ใบอนุมัติโครงงานปริญญานิพนธ์
สาขาวิชา วิทยาการคอมพิวเตอร์ ภาควิชาคอมพิวเตอร์
คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร

ชื่อปริญญานิพนธ์ การประยุกต์ใช้งานอัลกอริทึมเชิงพันธุกรรมในเครือข่ายสถานะสะท้อน
Application of the Genetic Algorithm in Echo State Networks

ผู้จัดทำ นาย วณิชช์ ตรีพิชพันธุ์

ปีการศึกษา 2567

โครงงานปริญญานิพนธนี้ได้รับการอนุมัติใหเป็นสวนหนึ่งของการศึกษาตาม
หลักสูตรวิทยาศาสตรบัณฑิต สาขาวิชา วิทยาการคอมพิวเตอร์
ภาควิชาคอมพิวเตอร์ คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร

..... ประธานกรรมการสอบ

(ผู้ช่วยศาสตราจารย์ ดร.ณัฐโชติ พรหมฤทธิ์)

..... กรรมการสอบ

(อาจารย์อภิเษก หงษ์วิทยากร)

..... อาจารยที่ปรึกษาและกรรมการสอบ

(ผู้ช่วยศาสตราจารย์ ดร.คทา ประดิษฐ์วงศ์)

การประยุกต์ใช้งานอัลกอริทึมเชิงพันธุกรรมในเครือข่ายสถานะสะท้อน

Application of the Genetic Algorithm in Echo State Networks

วณิชช์ ตรีพิชพันธุ์

ปริญญานิพนธนี้เป็นสวนหนึ่งของการศึกษาในหลักสูตรวิทยาศาสตรบัณฑิต

สาขาวิชา วิทยาการคอมพิวเตอร์ ภาควิชาคอมพิวเตอร์

คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร

ปีการศึกษา 2567

ชื่อปริญญานิพนธ์ การประยุกต์ใช้งานอัลกอริทึมเชิงพันธุกรรมในเครือข่ายสถานะสะท้อน
Application of the Genetic Algorithm in Echo State Networks
ผู้จัดทำ นาย วณิชช์ ตรีพิชพันธุ์
อาจารย์ที่ปรึกษา ผู้ช่วยศาสตราจารย์ ดร.คทา ประดิษฐ์วงศ์
ชื่อปริญญา วิทยาศาสตรบัณฑิต (วิทยาการคอมพิวเตอร์)
ปีการศึกษา 2567

บทคัดย่อ

โครงงานเรื่อง การประยุกต์ใช้งานอัลกอริทึมเชิงพันธุกรรมในเครือข่ายสถานะสะท้อน มุ่งเน้นในเรื่องการประเมินประสิทธิภาพของอัลกอริทึมและ echo state networks(ESN) โดย ESN จะเป็นการทำนายค่าจากดาต้าที่กำหนดและประเมินผลการทำนายโดยใช้ mean squared error (MSE) ซึ่งค่าที่นำมาประยุกต์ คือ ค่าขนาดจำนวน unit ของ reservoir ที่อยู่ใน ESN โดยค่าที่เหมาะสมของ reservoir ที่ต้องการคือ ค่าที่ทำให้ผลลัพธ์ mean squared error (MSE) พบค่าที่ดีที่สุดในขอบเขตของอัลกอริทึมเชิงพันธุกรรม โดยการหาค่า reservoir จะผ่านกระบวนการของ อัลกอริทึมเชิงพันธุกรรม โดยจะมีกระบวนการดังนี้ การสร้างประชากร การหาค่าที่เหมาะสม การคัดเลือก การไขว้เปลี่ยน และ การกลายพันธุ์

คำสำคัญ: อัลกอริทึมเชิงพันธุกรรม
Keyword: echo state networks (ESN), mean squared error (MSE)

กิตติกรรมประกาศ

โครงการปริญญานิพนธ์เรื่องการประยุกต์ใช้งานอัลกอริทึมเชิงพันธุกรรมในเครือข่ายสถานะสะท้อน สามารถสำเร็จลุล่วงไปได้ด้วยดี เนื่องจากได้รับการสนับสนุนทางด้านความรู้จากบุคคลที่ผู้จัดทำจะกล่าวถึงดังต่อไปนี้ ขอขอบพระคุณผู้ช่วยศาสตราจารย์ ดร. คทา ประดิษฐ์วงศ์ ผู้ให้คำปรึกษา ที่ได้ให้ความรู้และให้คำแนะนำในการศึกษาค้นคว้าตั้งแต่เริ่มจนปริญญานิพนธ์ฉบับนี้สำเร็จลุล่วงได้ดี ขอขอบพระคุณอาจารย์คณะกรรมการทุกท่าน สำหรับคำติชม ความรู้ที่ให้มาซึ่งได้นำมาปรับแก้ไขให้วิทยานิพนธ์ดีขึ้นจนปริญญานิพนธ์นี้สำเร็จลุล่วง ขอกราบขอบพระคุณคณาจารย์มหาวิทยาลัยศิลปากร คณะวิทยาศาสตร์ และภาควิชาคอมพิวเตอร์ที่ให้วิชาความรู้และสถานที่ในการจัดทำโครงงานปริญญานิพนธ์

สุดท้ายนี้ขอกราบขอบพระคุณบิดา มารดา และครอบครัวที่ให้กำลังใจและสนับสนุนในทุกเรื่อง ขอบคุณเพื่อน พี่ น้อง และผู้เกี่ยวข้องทุกท่านที่มีส่วนร่วมในการช่วยเหลือให้คำแนะนำและเป็นกำลังใจอย่างดี ผู้จัดทำขอกราบขอบพระคุณเป็นอย่างสูงไว้ ณ โอกาสนี้

วณิชช์ ตรีพิชพันธุ์

สารบัญ

หน้า

บทคัดย่อ ..... ก

กิตติกรรมประกาศ ..... ข

สารบัญ ..... ค

สารบัญภาพ ..... จ

บทที่ 1 บทนำ 1

1.1 ที่มาและความสำคัญของปัญหา 1

1.2 วัตถุประสงค์ 1

1.3 ลักษณะและขอบเขต 2

1.4 อุปกรณ์และเครื่องมือที่ใช้ 2

1.5 ผลที่คาดว่าจะได้รับ 2

บทที่ 2 ผลงานที่เกี่ยวข้อง 3

2.1 Hybrid Structure Based PSO for ESN Optimization. [1] 3

2.2 Optimization of echo state network behavior space based on microbial genetic algorithm. [2] 5

บทที่ 3 ทฤษฎีและความรู้ที่เกี่ยวข้อง 7

3.1 Echo State Network [3] 7

3.2 Python [4] 11

3.3 Visual Studio Code [5] 11

3.4 numpy [6] 11

3.5 genetic algorithm [7] 12

3.6 Recurrent Neural Network (RNN) [8] 13

บทที่ 4 ขั้นตอนและแผนการดำเนินงาน 15

4.1 ขั้นตอนการดำเนินงาน 15

4.2 แผนการดำเนินงาน 16

บทที่ 5 วิธีการดำเนินงาน 17

5.1 โครงสร้างหรือภาพรวมของโปรแกรม 17

5.2 ขั้นตอนการทำงานและโครงสร้างของโปรแกรม 17

5.3 ขั้นตอนการทำงานของโปรแกรม 18

5.4 ขั้นตอนในการทดลอง 30

mackeyGlass Time Series 30

HENON.DAT 31

ROSSLER.DAT 32

LORENZ.DAT 33

ขอบเขตการทดลอง 34

5.5 การวิเคราะห์ผลของการทดลอง 35
บทที่ 6 ผลการดำเนินงาน ข้อจำกัด และข้อเสนอแนะ..... 36
6.1 ผลการดำเนินงาน 36
6.1.1 ดาต้าที่ใช้ทดสอบ 36
6.1.2 การเก็บผลแบบตาราง 42
6.1.3 สรุปผลการทดลอง 44
6.2 ข้อจำกัด.....47
6.3 ข้อเสนอแนะ.....47
บรรณานุกรม..... 48
ภาคผนวก ก การทำงานของ Genetic Algorithm..... 49

สารบัญภาพ

หน้า

รูป 1 กราฟการเปรียบเทียบ PREDICTION ERROR ระหว่าง OESN กับ HPSO 4
รูป 2 รูปตัวอย่างของ ECHO STATE NETWORK 7
รูป 3 PYTHON 11
รูป 4 VISUAL STUDIO CODE 11
รูป 5 การทำงานของ GENETIC ALGORITHM 12
รูป 6 RECURRENT NEURAL NETWORK 13
รูป 7: โครงสร้างการทำงานของโปรแกรม 17
รูป 8 กราฟแสดงตัวอย่างข้อมูลใน MACKEYGLASS_T17.TXT ขนาด 2000 ตัว 31
รูป 9 กราฟแสดงตัวอย่างข้อมูลใน MACKEYGLASS_T30.TXT ขนาด 2000 ตัว 31
รูป 10 กราฟแสดงตัวอย่างข้อมูลใน HENON.DAT ขนาด 300 ตัว 32
รูป 11 กราฟแสดงตัวอย่างข้อมูลใน ROSSLER.DAT ขนาด 2000 ตัว 33
รูป 12 กราฟแสดงตัวอย่างข้อมูลใน LORENZ.DAT ขนาด 2000 ตัว 34
รูป 13 กราฟของ ORIGINAL ESN, RESERVOIR SIZE = 1000 44
รูป 14 กราฟของ GA-ENS, RESERVOIR SIZE = 744 45
รูป 15 กราฟของ GA-ESN ,RESERVOIR SIZE= 1000 45
รูป 16 ORIGINAL ESN ,RESERVOIR SIZE= 1 46
รูป 17 กราฟ ORIGINAL ESN, RESERVOIR SIZE = 1000 46
รูป 18 กราฟ GA-ESN, RESERVOIR = 671 47
รูปผนวก ที่ ก 1 ไฟล์ .ENV 49
รูปผนวก ที่ ก 2 ฟังก์ชัน MAIN 50
รูปผนวก ที่ ก 3 ฟังก์ชัน __INIT__(SELF) 50
รูปผนวก ที่ ก 4 ฟังก์ชัน POPSIZE(SELF) และ ฟังก์ชัน LENGTH(SELF) 51
รูปผนวก ที่ ก 5 ฟังก์ชัน RUN_GENERATION(SELF,T) 1 51
รูปผนวก ที่ ก 6 ฟังก์ชัน CHROMOSOME(SELF) 52
รูปผนวก ที่ ก 7 ฟังก์ชัน DECODE(SELF,A) 53
รูปผนวก ที่ ก 8 ฟังก์ชัน SENDRESSIZE(SELF) 53
รูปผนวก ที่ ก 9 ฟังก์ชัน MSE(SELF) 54
รูปผนวก ที่ ก 10 ฟังก์ชัน RUN_GENERATION(SELF,T) 2 54
รูปผนวก ที่ ก 11 ฟังก์ชัน NORMALIZATION(SELF) 55
รูปผนวก ที่ ก 12 ฟังก์ชัน ROULETTESELECTION(SELF) 55

รูปผนวก ที่ ก 13 ฟังก์ชัน CLONING(SELF) 56
รูปผนวก ที่ ก 14 ฟังก์ชัน CROSSOVER(SELF) 57
รูปผนวก ที่ ก 15 ฟังก์ชัน MUTATION(SELF) 57
รูปผนวก ที่ ก 16 ฟังก์ชัน DECODEMUTATION(SELF,A) 58
รูปผนวก ที่ ก 17 ฟังก์ชัน SENDRESSIZEMU 59
รูปผนวก ที่ ก 18 CHECKGEN.PY 60
รูปผนวก ที่ ก 19 ลูปในการเช็คค่า RESERVOIR 61
รูปผนวก ที่ ก 20 เงื่อนไขในการเช็คหาค่า INF 61
รูปผนวก ที่ ก 21 FINDBESTFOR30.PY 62
รูปผนวก ที่ ก 22 หาค่าสถิติโดยใช้ NUMPY 62

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญของปัญหา

การฝึก Recurrent Neural Networks (RNNs) เป็นเรื่องที่มีความซับซ้อนโดยธรรมชาติ ซึ่งทำให้นักพัฒนาหลายคนหลีกเลี่ยงที่จะใช้งานโมเดลประเภทนี้ อย่างไรก็ตาม RNNs เป็นเครื่องมือที่มีศักยภาพสูง โดยสามารถจดจำข้อมูลในอดีตได้อย่างยาวนานและมีความสามารถในการคำนวณที่ยืดหยุ่น หลายสิบปีก่อนหน้า ได้มีการเสนอวิธีการใหม่ในการทำความเข้าใจ ฝึกฝน และใช้งาน RNNs นั่นคือ Echo State Network (ESN) ในสาขา Machine Learning และ Liquid State Machines (LSMs) ใน Computational Neuroscience โดย ESNs มีแนวคิดหลักคือ การสุ่มสร้าง RNN ที่เรียกว่า reservoir และทำการฝึกเฉพาะส่วนที่เรียกว่า readout ซึ่งเป็นชั้นที่เชื่อมต่อกับผลลัพธ์เท่านั้น แม้ว่าแนวคิดนี้จะดูเรียบง่าย แต่กลับให้ผลลัพธ์ที่ดีในงานหลากหลายประเภท

ESN มีจุดเด่นที่ความเรียบง่ายและค่าใช้จ่ายในการคำนวณต่ำ แต่อาจเกิดข้อผิดพลาดได้ ดังนั้นการใช้งาน ESN อย่างมีประสิทธิภาพต้องอาศัยประสบการณ์ในการตั้งค่าพารามิเตอร์ต่าง ๆ อย่างเหมาะสม

โดยโมดูลที่เรานำมาใช้นั้นชื่อว่า minimalESN ซึ่งเป็นโมดูลของ Mantas Lukoševičius ซึ่งได้ทำการเขียนวิธีการใช้งานของ Echo State Network (ESN) แบบง่ายเอาไว้เพื่อเอื้อต่อการศึกษาและนำไปพัฒนาต่อ แต่ยังมีบางพารามิเตอร์ที่อยากจะนำไปปรับและนำมาประยุกต์กับอัลกอริทึมเชิงพันธุกรรม (Genetic Algorithm) เพื่อเพิ่มประสิทธิภาพการทำนาย โดยโมดูลนี้นั้นจะเรียนรู้การทำนาย จากดาต้า ของ Mackey-Glass chaotic time series (delay=17) ซึ่งเป็นดาต้าที่ใช้กันแพร่หลายสำหรับการทดสอบ machine learning (ML) โมดูล

เราได้เล็งเห็นว่าสามารถนำไปต่อยอดในการเสริมความแม่นยำในการทำนายโดยนำ อัลกอริทึมเชิงพันธุกรรม (Genetic Algorithm) ไปประยุกต์ปรับใช้ในโมดูลของ minimalESN ซึ่งเป็นอัลกอริทึมที่ใช้ในการแก้ปัญหาการหาค่าที่เหมาะสมหรือการค้นหาคำตอบในลักษณะการเรียนรู้จากกระบวนการส่งต่อพันธุกรรมของสิ่งมีชีวิต การพัฒนาขึ้นจากหลักการของทฤษฎีการวิวัฒนาการของชาร์ลส์ดาร์วิน เป็นการจำลองกระบวนการคัดเลือกทางธรรมชาติ และการกลายพันธุ์เพื่อปรับปรุงคุณภาพของประชากรให้มีศักยภาพใกล้เคียงกับคำตอบที่เหมาะสมหรือเหนือกว่า ซึ่งอัลกอริทึมเชิงพันธุกรรม (Genetic Algorithm หรือ GA) สามารถนำไปประยุกต์ใช้ในการแก้ปัญหาการหาค่าที่เหมาะสมหรือการปรับปรุงข้อมูลต่าง ๆ เช่น การปรับค่าพารามิเตอร์ต่าง ๆ หรือ งานที่เกี่ยวข้องกับ machine learning เป็นต้น

รวมถึงมีการเพิ่มดาต้า ต่าง ๆ เข้าไปที่นอกเหนือจาก Mackey-Glass เพื่อเพิ่มความสามารถหรือประสิทธิภาพในการทำนายของโครงงานนี้

1.2 วัตถุประสงค์

1.2.1 สามารถทำนายผลความแม่นยำของดาต้าในโมดูลของ minimalESN โดยนำ อัลกอริทึมเชิงพันธุกรรม (Genetic Algorithm) มาเพิ่มประสิทธิภาพการทำนาย

1.2.2 เพื่อประเมินประสิทธิภาพของอัลกอริทึมเชิงพันธุกรรม (Genetic Algorithm) ที่นำมาใช้ร่วมกับ ESN

1.3 ลักษณะและขอบเขต

1.3.1 นำ Genetic Algorithm มาประยุกต์กับโมดูล minimalESN

1.3.2 สามารถเพิ่มดาต้านอกเหนือ Mackey-Glass ได้

1.4 อุปกรณ์และเครื่องมือที่ใช้

1.4.1 ฮาร์ดแวร์

คอมพิวเตอร์ที่ใช้ในการพัฒนา

- CPU: i7-11700F
- GPU: RTX3060ti
- Memory: 32GB

1.4.2 ซอฟต์แวร์

- ระบบปฏิบัติการวินโดว์ 11
- โปรแกรม Anaconda Navigator โดยใช้ vs code
- minimalESN Package

1.5 ผลที่คาดว่าจะได้รับ

1.5.1 สามารถเพิ่มประสิทธิภาพการทำนายของโมดูล ESN ที่นำมาประยุกต์ได้

1.5.2 สามารถใช้ดาต้านอกเหนือจาก Mackey-Glass ได้

บทที่ 2 ผลงานที่เกี่ยวข้อง

ในบทนี้จะกล่าวถึงงานวิจัยต่าง ๆ ที่มีความใกล้เคียงกับโครงงานนี้ ที่นำมาอ้างอิงในการศึกษาหาความรู้ ซึ่งจะประกอบด้วยเนื้อหาโดยสรุปโดยรวมของแต่ละงานวิจัย

2.1 Hybrid Structure Based PSO for ESN Optimization. [1]

ประสิทธิภาพของ Echo state network (ESN) มีความเกี่ยวข้องกับ output weights (น้ำหนักเอาต์พุต) ดังนั้นการหาค่า output weights ที่ดีมีความสำคัญเป็นอย่างมาก และมีหลายหลายวิธีด้วยกัน หนึ่งในนั้น the particle swarm optimization (PSO) algorithm เป็นวิธีที่ใช้กันอย่างแพร่หลาย ซึ่งแนวคิดพื้นฐานของ PSO คือการค้นหาวิธีแก้ปัญหาที่ดีที่สุดโดยความร่วมมือของแต่ละ particle ในกลุ่ม แต่สิ่งที่จะเพิ่มมาใช้ในงานวิจัยนี้คือ HPSO (Hybrid Particle Swarm Optimization) เป็นรูปแบบการปรับปรุงของอัลกอริทึม Particle Swarm Optimization (PSO) โดยเพิ่มโครงสร้างการทำงานแบบผสม (Hybrid Topology) เพื่อเพิ่มประสิทธิภาพในการค้นหาคำตอบที่เหมาะสมที่สุดได้ดียิ่งขึ้น โดยงานวิจัยนี้จะเปรียบเทียบกับ original ESN

ใน HPSO (Hybrid Particle Swarm Optimization) อนุภาคแต่ละตัวแทนค่าที่เป็นไปได้ของเมตริกซ์น้ำหนักผลลัพธ์ (Wout) ของ Echo State Network (ESN) จากนั้นเพื่อประเมินประสิทธิภาพของอนุภาคแต่ละตัว จะใช้ค่า Root Mean Square Error (RMSE) เป็นตัวชี้วัด โดยมีสมการดังนี้

$$RMSE = \sqrt{\sum_{k=1}^{L} \frac{(t(k) - y(k))^2}{L}} \qquad (1)$$

โดย L เป็นจำนวนของการ training t(k) คือ ตำแหน่งของเป้าหมาย, y(k) คือ network output ซึ่งค่า RMSE ยิ่งน้อยยิ่งดี

ขั้นตอนในการทำ HPSO นั้นมีอยู่ 4 ขั้นตอนด้วยกันคือ

1. สร้างเมทริกซ์ น้ำหนักเริ่มต้นแบบสุ่ม
    - สร้างเมทริกซ์ น้ำหนักอินพุต (Win) และน้ำหนักของ Reservoir (W) แบบสุ่ม
    - Win เชื่อมโยงสัญญาณอินพุตกับ Reservoir
    - W แสดงการเชื่อมโยงภายในของ Reservoir

2. เก็บสถานะค่า reservoir

เก็บจากสมการ

$$\mathbf{x}(k) = \mathbf{g}(\mathbf{W}\mathbf{x}(k-1) + \mathbf{W}^{in}\mathbf{u}(k)) \qquad (2)$$

3. คำนวณผลลัพธ์ Wout ด้วย RMSE

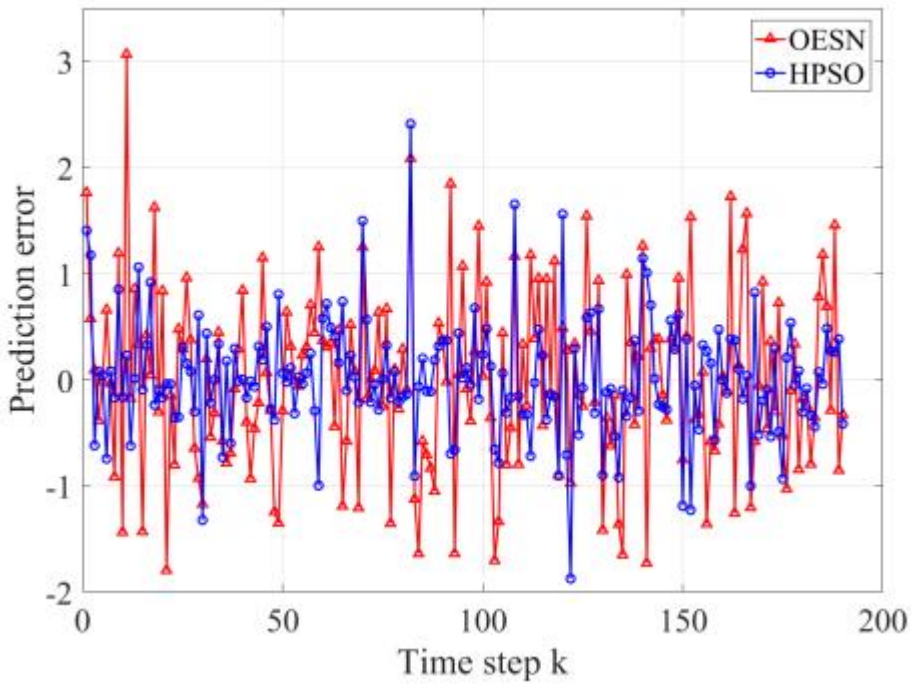
$$RMSE = \sqrt{\sum_{k=1}^{L} \frac{(t(k) - y(k))^2}{L}}$$

4. ทดสอบ HPSO-ESN

- ใช้น้ำหนักผลลัพธ์ (Wout) ที่ได้จาก HPSO ในขั้นตอนก่อนหน้า
- ทดสอบ ESN บนชุดข้อมูลใหม่ (Test Data) เพื่อประเมินประสิทธิภาพการพยากรณ์

ผลเปรียบเทียบการทำนายระหว่าง original ESN (OESN) กับ HPSO-ESN ผ่านการพยากรณ์ค่าความเข้มข้นของ **NH4-N (แอมโมเนียไนโตรเจน)**

Methods	N	Training time (s)	Training RMSE (Mean)	Training RMSE (Std)	Testing RMSE (Mean)	Testing RMSE (Std)
OESN	100	1.428	0.5618	0.06845	0.8022	0.0724
HPSO-ESN	100	32.183	0.3946	0.0338	0.5506	0.0351



รูป 1 กราฟการเปรียบเทียบ Prediction error ระหว่าง OESN กับ HPSO

โดยจะเห็นได้ว่าในกราฟการทำนายระหว่าง OESN กับ HPSO ค่าความทำนายความผิดพลาดของ HPSO จะน้อยกว่า OESN เป็นอย่างมากจึงสรุปได้ว่า HPSO-ESN สามารถทำนายได้ดีกว่า OESN

2.2 Optimization of echo state network behavior space based on microbial genetic algorithm [2]

โดยงานวิจัยนี้เห็นว่าในการทำงานประยุกต์ใช้ส่วนใหญ่ของ Echo state network จะมีการกำหนดค่า reservoir มาโดยการสุ่ม ซึ่งอาจจะทำให้เกิด ค่าที่ไม่เสถียรของตัวแปรบางตัว ทำให้คนใช้เวลาส่วนใหญ่ในการปรับค่าพารามิเตอร์ให้คงที่ โดยงานวิจัยนี้จะเสนอเรื่องของการหาค่าพารามิเตอร์ reservoir ให้เหมาะสม

Microbial Genetic Algorithm (MGA) จะมีกระบวนการดังนี้ การสร้างประชากร, การไขว้เปลี่ยน, การกลายพันธุ์ และการคัดเลือก โดยจะนำกระบวนการเหล่านี้มาปรับใช้กับค่า reservoir และผลลัพธ์ของการฝึกการทดสอบของโมเดลนี้คือค่า Normalized Mean Squared Error (NMSE) โดยเกิดจาก

$$NMSE(y, y^*) = \frac{\left\| y - y^* \right\|^2}{\left\| y^* \right\|^2} \qquad (3)$$

โดย y คือ network output และ y* คือ expected output หรือค่าที่คาดหวังเอาไว้

เป้าหมายของการทดสอบของงานวิจัยนี้มีอยู่ 2 ดาต้าด้วยกันคือ

1. NARMA-10 เป็นการจำลองค่าที่มีความต่อเนื่องกัน n ค่า และเป็น highly nonlinear dynamic โดยมีสมการดังนี้

$$y(t+1) = \gamma(\alpha y(t) + \beta y(t) \sum_{i=0}^{n-1} y(t-i) + 1.5u(t-9)u(t) + \delta) \qquad (4)$$

โดยจะทำการสร้างค่า 5600 ค่า และแบ่งไป training 2800 ค่า และ testing 2800

2. Laser timing prediction เป็นดาต้าที่ถูกบันทึกจาก infrared laser ใน chaotic state โดยในดาต้านี้ มีค่าอยู่ด้วยกัน 10,093 ค่า โดยจะนำ 8300 ค่า ไปใช้สำหรับการ training และส่วนที่เหลือคือนำไป testing

ผลการทดลอง

งานวิจัยนี้ทำการทดสอบกับ วิธีที่มีอยู่แล้วโดยมี the optimization ESN with a binary PSO (BPSO-ESN) และ Growing ESN (GESN) โดยจะเซ็ตค่า reservoir ทั้งหมด N ค่า โดยจะมี N = 200,300,500 และจะทำการรัน 30 ครั้งของแต่ละการสร้างประชากรเริ่มต้นโดยแสดงผลลงตารางดังนี้

Dataset	models	N=200	N=300	N=500
NARMA -10	MGA-ESN(300)	0.0310	0.0143	0.0140
	MGA-ESN(600)	0.0240	0.0133	0.0139
	MGA-ESN(1200)	**0.0173**	0.0112	0.0105
	BPSO-ESN	0.0428	0.0369	0.0346
	GESN	0.0240	0.0107	0.0049
	PSO-GESN	0.0208	**0.0101**	**0.0047**

Dataset	models	N=200	N=300	N=500
Laser	MGA-ESN(300)	0.0048	0.0037	0.0037
	MGA-ESN(600)	0.0045	0.0035	0.0033
	MGA-ESN(1200)	**0.0039**	**0.0033**	**0.0032**
	BPSO-ESN	0.0143	0.0142	0.0140
	GESN	0.0059	0.0045	0.0037
	PSO-GESN	0.0056	0.004	0.0033

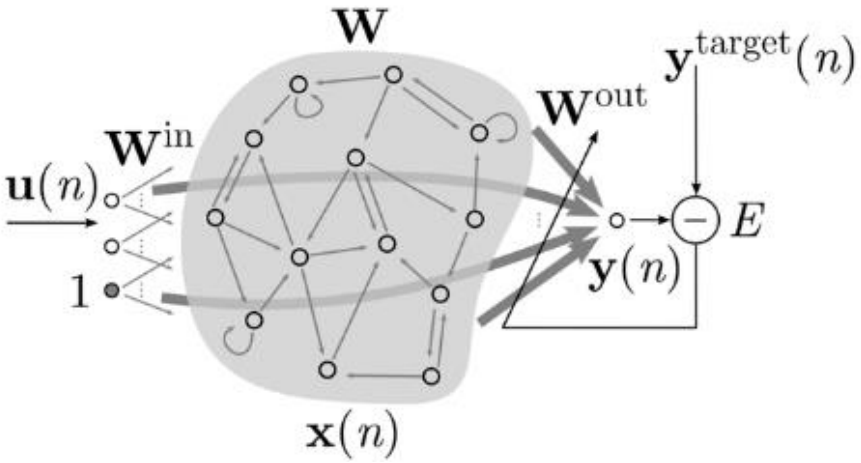
โดยจากการทดลองโดยใช้ ดาต้า NARMA-10 และ Laser แสดงให้เห็นว่าค่า NMSE ของ MGA-ESN มีค่าที่ดีกว่าหรือค่าที่น้อยกว่า ดาต้า BPSO-ESN, GESN, PSO-GESN

โดยงานวิจัยของ Hybrid Structure Based PSO for ESN Optimization และ Optimization of echo state network behavior space based on microbial genetic algorithm สองงานวิจัยนี้จะมาเป็นต้นแบบในการออกแบบในการออกแบบการทดลองและเปรียบเทียบ ในการนำ genetic algorithm (GA) มาประยุกต์ เช่น การออกแบบการเปรียบเทียบระหว่าง original ESN กับ GA-ESN หรือ การใช้หลายดาต้าเซ็ตมาเปรียบเทียบกัน

บทที่ 3 ทฤษฎีและความรู้ที่เกี่ยวข้อง

ในบทนี้จะกล่าวถึงทฤษฎีและความรู้ที่เกี่ยวข้องเพื่อนำมาใช้ในการทำโครงงาน การประยุกต์ใช้งานอัลกอริทึมเชิงพันธุกรรมในเครือข่ายสถานะสะท้อน

3.1 Echo State Network [3]



รูป 2 รูปตัวอย่างของ echo state network

Echo State Network (ESN) หรือเครือข่ายสถานะสะท้อน มีองค์ประกอบด้วยกัน 3 ส่วนด้วยกันคือ

1. Input layer ประกอบไปด้วย
   - $u(n)$ เป็น data input หรือ ดาต้าอินพุต $u(n) \in \mathbb{R}^{N_u} \ : n = 1,..,T$ โดย n เป็นตำแหน่งของช่วงเวลาและ T เป็นจำนวนตำแหน่งของดาต้าเซ็ต
   - Win เป็น Weight input หรือ น้ำหนัก input

```
Win = (np.random.rand(resSize,1+inSize) - 0.5) * 1
```

โดยอ้างอิงจากโค้ดเกิดจากการสุ่มค่าระหว่าง 0 ถึง 1 เก็บมาเป็นอาเรย์สองมิติที่มีขนาด row = ขนาดของ reservoir และ column = 1+ขนาดของ input layer โดยจะนำค่าทั้งหมดในอาเรย์มาลบด้วย 0.5 ท้ายที่สุด

2. Reservoir

ความสำคัญของขนาดจำนวน unit ใน reservoir โดยทั่ว ๆ ไปยิ่งค่า unit ใน reservoir มากเท่าไร ความสามารถในการเก็บค่าสถานะยิ่งมากเท่านั้น ซึ่งทำให้หน่วยความจำในการเก็บค่าของ $x(n)$ มากขึ้นด้วยยิ่งทำให้สามารถหา Linear Combination ที่ใกล้เคียงกับค่าของผลลัพธ์ที่คาดหวังไว้ได้ ทำให้การหาค่าจำนวน unit reservoir เหมาะสมอาจจะทำ

ให้การทำนายดีขึ้นได้เช่นเดียวกัน โดย $x(n)$ จะเป็นการเก็บข้อมูลสถานะของแต่ละโนดใน reservoir ณ เวลาที่ n เป็นเวกเตอร์

องค์ประกอบของ reservoir

- W เป็นค่า Weight หรือค่าน้ำหนักที่อยู่ใน reservoir

```
W = np.random.rand(resSize,resSize) - 0.5
```

โดยอ้างอิงจากโค้ดเกิดจากการสุ่มค่าระหว่าง 0 ถึง 1 เก็บเป็นอาเรย์สองมิติที่มีขนาด แถวและคอลัมน์เท่ากับขนาดของ reservoir โดยในโค้ดถูกแทนด้วยตัวแปร resSize โดยจะนำค่าทั้งหมดในอาเรย์ มาลบด้วย 0.5 ท้ายที่สุดและจะมีการ normalize ค่า W

```
rhoW = max(abs(linalg.eig(W)[0]))
W *= 1.25 / rhoW
```

โดยจะนำค่าใน W ทั้งหมดไปคูณกับค่า 1.25 ที่หารด้วย $rhow$ ที่เกิดจากค่าที่มากที่สุด (max)ของค่าสัมบูรณ์ (abs) ในฟังก์ชัน $eigenvalues$ ที่คำนวนจาก W ในตำแหน่งที่ 0

- $x(n)$ คือค่าที่เกิดจากการเก็บค่าสถานะของ data input และ weight input โดยมีสมการดังนี้

$$\tilde{x}(n) = \tanh\left(W^{in}[1; u(n)] + Wx(n-1)\right) \qquad (5)$$

$$x(n) = (1-\alpha)x(n-1) + \alpha\tilde{x}(n) \qquad (6)$$

โดย α คือค่า leaking rate เป็นค่าระหว่าง 0 ถึง 1

$\tilde{x}(n)$ เกิดจากฟังก์ชัน $\tanh($ การคูณกันระหว่าง W^{in} กับ $[1; u(n)])$ โดย $[1; u(n)]$ คือการนำค่าคงที่ 1 และ $u(n)$ มาต่อแถวกันในแนวตั้งหรือเพิ่มแถวเข้าด้วยกันจะได้ $\begin{bmatrix} 1 \\ u(n) \end{bmatrix}$ และ บวกกับการคูณกันของ W และ $x(n-1)$ ซึ่ง $n-1$ คือตำแหน่งเวลาสถานะก่อนหน้านั้นเอง

และ $x(n)$ จะเป็นฟังก์ชันในการเก็บสถานะเกิดจาก 1 ลบกับค่า α คูณกับ $x(n-1)$ หรือก็คือ x ในสถานะก่อนหน้าและบวกกับการคูณกันระหว่าง α และ $\tilde{x}(n)$

```
45  # run the reservoir with the data and collect X
46  x = np.zeros((resSize,1))
47  for t in range(trainLen):
48      u = data[t]
49      x = (1-a)*x + a*np.tanh( np.dot( Win, np.vstack((1,u)) ) + np.dot( W, x ) )
50      if t >= initLen:
51          X[:,t-initLen] = np.vstack((1,u,x))[:,0]
```

อ้างอิงจากโค้ด ในบรรทัดที่ 49 จะนำสมการที่ (5) ไปแทนใน (6)

โดย x เริ่มต้นนั้นเกิดจากการสร้างอาเรย์สองมิติที่เก็บค่า 0 ทั้งหมดโดยมีขนาดแถวเท่ากับ resSize และคอลัมน์เท่ากับ 1 จากนั้นจะเข้าลูปในการอัพเดทสถานะของ x โดยจะมีลูปตามขนาดของการ train ที่กำหนด โดยเริ่มเก็บ data input ตั้งแต่ตำแหน่งตัวที่ 0 ของ data และทำการเก็บสถานะ โดยหลังการทำเสร็จหนึ่งรอบเราจะนำสถานะ

x ก่อนหน้ามาใช้กับสถานะปัจจุบันและอัพเดทสถานะใน x เข้าไปใหม่ทำแบบนี้จนตำแหน่งของลูปถึง ค่า $trainLen - 1$ เมื่อตำแหน่งลูปในโค้ดนั้นมากกว่าหรือเท่ากับ initLen จะทำการเก็บสถานะของ X

```
X = np.zeros((1+inSize+resSize,trainLen-initLen))
# set the corresponding target matrix directly
```

โดย X เริ่มต้นก่อนหน้าเกิดจากการสร้างค่าอาเรย์สองมิติโดยมีขนาดแถวเท่ากับ 1+inSize + resSize และคอลัมน์เท่ากับ trainLen – initLen

```
    if t >= initLen:
        X[:,t-initLen] = np.vstack((1,u,x))[:,0]
```

เราจะทำการอัพเดท ค่า X ในทุกตัวในคอลัมน์ตำแหน่งที่ $t - initLen$ โดยเก็บค่าจาก การดึงเอาคอลัมน์แรก$[:,0]$ หลังนำค่าคงที่ $1, u$ และ x มาต่อกันในแถวจากคำสั่ง $np.vstack$ โดยจะมีลักษณะหน้าตา $\begin{bmatrix} 1 \\ u \\ x \end{bmatrix}$ เป็นต้น และทำการลูปแบบนี้ตั้งแต่ $t = intiLen$ ถึง $trainLen - 1$

3. Output layer

Output layer ทำหน้าที่สำคัญในการสร้างผลลัพธ์สุดท้าย $y(n)$ จากข้อมูลที่ได้จาก reservoir $x(n)$, อินพุต $u(n)$ และ $Wout$ หรือ weight output โดยมีลักษณะการทำงานเฉพาะตัวในรูปแบบเชิงเส้น (Linear Readout Layer) โดยมีสมการ

$$y(n) = Wout[1; u(n)x(n)] \qquad (7)$$

โดย $[1; u(n)x(n)]$ จะเป็นการนำแต่ละตัวแปรมาต่อแถวกันโดนมี 1 เป็นค่าคงที่ $u(n)$ เป็นดาต้าอินพุต ณ เวลา n และ $x(n)$ เป็นสถานะของ reservoir โดยจะนำรวมกันได้เป็นเวกเตอร์ $\begin{bmatrix} 1 \\ u(n) \\ x(n) \end{bmatrix}$

จากสมการที่ (4) เราสามารถเขียนในระบบของเมตริกซ์ได้เป็น

$$Y = Wout\ X \qquad (8)$$

โดย Y คือค่าทั้งหมดของ $y(n)$ และ X เป็นค่าทั้งหมดของ $[1; u(n)x(n)]$ เพื่อความสะดวกในการใช้สัญลักษณ์

การหาค่าที่เหมาะสมของค่า $Wout$ เพื่อที่จะทำให้ ความผิดพลาดของการทำนายระหว่าง $y(n)$ และ $y^{target}(n)$ น้อยที่สุด และสามารถหาค่า $Wout$ เขียนในรูปแบบเชิงเส้นได้จาก

$$Y^{target} = Wout\ X \qquad (9)$$

โดย Y คือค่าทั้งหมดของ $y(n)$ และค่า $Wout$ สามารถพิจารณาโดยใช้เมตริกซ์ในการแก้สมการเชิงเส้นในสมการที่ (5) จะได้เป็น

$$Wout = Y^{target} X^T (XX^T + \beta I)^{-1} \qquad (10)$$

โดย Y^{target} เป็นผลลัพธ์เป้าหมาย X^T คือทรานสโพสของ X ,β คือค่า regularization และ I เป็นเมตริกซ์เอกลักษณ์

อ้างอิงจากโค้ดในการหา $Wout$

```
Wout = linalg.solve( np.dot(X,X.T) + reg*np.eye(1+inSize+resSize),
    np.dot(X,Yt.T) ).T
```

ในส่วนของโค้ดใช้ฟังก์ชันของ linalg.solve ในการแก้ไขค่า $Wout$ ในสมการที่ (6)

โดยหลังการหาค่าของ $Wout$ ได้สำเร็จจะไปยังส่วนของการ testing เพื่อหาค่า Y จาก $y(n)$ ทั้งหมด

อ้างอิงจากโค้ดในส่วนของการ testing

```
66  Y = np.zeros((outSize,testLen))
67  u = data[trainLen]
68  for t in range(testLen):
69      x = (1-a)*x + a*np.tanh( np.dot( Win, np.vstack((1,u)) ) + np.dot( W, x ) )
70      y = np.dot( Wout, np.vstack((1,u,x)) )
71      Y[:,t] = y
72      # generative mode:
73      u = y
```

โดยจะสร้างอาเรย์ของ Y ที่มีขนาดแถวเท่ากับขนาดของเอาท์พุตไซซ์และคอลัมน์เท่ากับจำนวนที่ต้องการ testing และกำหนดตัวเริ่มต้นของอินพุตดาต้าตำแหน่งที่ต่อจากหลังการ training จากนั้นจะเข้าลูปในส่วนของการ test โดยจะเป็นลูปขนาดจำนวนที่ต้องการ testing เริ่มจากการเก็บค่าสถานะของ x จากสมการที่ (5) ไปแทนใน (6) ส่วนค่าที่ถูกอัพเดทตลอด ณ ช่วงเวลา t คือค่าของดาต้าอินพุต (หรือค่า u ในบรรทัดที่ 73) แทนที่ด้วยค่า $y(n)$ และตลอด ณ ช่วงเวลา t จะเก็บค่า $y(n)$ ทั้งหมดเข้าไปยังตัวแปร Y โดยจะเก็บค่าจากทุกตัวในแถวที่คอลัมน์ t

หลังทำการ test เสร็จแล้วนั้นเราก็จะหาค่า Mean squared error (MSE) โดยจะเปรียบระหว่าง $y(n)$ และ $y^{target}(n)$ โดยหาจากสมการ

$$MSE = \frac{1}{T}\sum_{n=1}^{T}(y(n) - y^{target}(n))^2 \qquad (11)$$

โดยค่า MSE นั้นยิ่งน้อยยิ่งทำให้รู้ว่าความผิดพลาดของการทำนายนั้นน้อยลงนั้นเอง

3.2 Python [4]

เป็นอินเทอร์พรีเตอร์ภาษาระดับสูงซึ่งสร้างโดย Guido van Rossum โดยเริ่มใน พ.ศ. 2533 การออกแบบของภาษา Python มุ่งเน้นให้ผู้โปรแกรมสามารถอ่านชุดคำสั่งได้โดยง่ายผ่านการใช้งานอักขระเว้นว่าง (whitespaces) จำนวนมาก นอกจากนั้นการออกแบบภาษา Python และการประยุกต์ใช้แนวคิดการเขียนโปรแกรมเชิงวัตถุในตัวภาษายังช่วยให้นักเขียนโปรแกรมสามารถเขียนโปรแกรมที่เป็นระเบียบ อ่านง่าย มีขนาดเล็ก และง่ายต่อการบำรุง

รูป 3 python

3.3 Visual Studio Code [5]

เป็นโปรแกรมแก้ไขซอร์สโค้ดที่พัฒนาโดยไมโครซอฟท์สำหรับ Windows, Linux และ macOS มีการสนับสนุนสำหรับการดีบัค การควบคุม Git ในตัวและ GitHub การเน้นไวยากรณ์ การเติมโค้ดอัจฉริยะ ตัวอย่าง และ code refactoring มันสามารถปรับแต่งได้หลายอย่าง ให้ผู้ใช้สามารถเปลี่ยนธีม แป้นพิมพ์ลัด การตั้งค่า และติดตั้งส่วนขยายที่เพิ่มฟังก์ชันการทำงานเพิ่มเติม ซอร์สโค้ดนั้นฟรีและโอเพนซอร์สและเผยแพร่ภายใต้สิทธิ์การใช้งาน MIT ไบนารีที่คอมไพล์แล้วเป็นฟรีแวร์และฟรีสำหรับการใช้ส่วนตัวหรือเพื่อการค้า

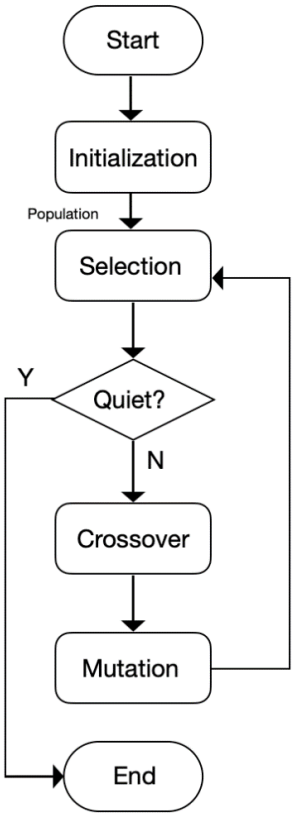
รูป 4 Visual Studio Code

3.4 numpy [6]

เป็น library ที่ import เข้ามาเพื่อเพิ่มการสนับสนุนสำหรับอาร์เรย์และเมทริกซ์ขนาดใหญ่หลายมิติพร้อมด้วยชุดฟังก์ชันทางคณิตศาสตร์ระดับสูงจำนวนมาก เพื่อดำเนินการกับอาเรย์เหล่านี้

3.5 genetic algorithm [7]

Genetic Algorithms เป็นหนึ่งในอัลกอริทึมที่เอาไว้หาค่าที่เหมาะสมที่สุด และ เรียนรู้จากกระบวนการส่งต่อพันธุกรรมของสิ่งมีชีวิต พัฒนาขึ้นจากหลักการของทฤษฎีการวิวัฒนาการของชาร์ลส์ดาร์วิน เป็นการจำลองกระบวนการคัดเลือกทางธรรมชาติ และการกลายพันธุ์เพื่อปรับปรุงคุณภาพของประชากรให้มีศักยภาพใกล้เคียงกับคำตอบที่เหมาะสมหรือเหนือกว่า ซึ่งอัลกอริทึมเชิงพันธุกรรม (Genetic Algorithm หรือ GA) สามารถนำไปประยุกต์ใช้ในการแก้ปัญหาการหาค่าที่เหมาะสมหรือการปรับปรุงข้อมูลต่าง ๆ เช่น การปรับค่าพารามิเตอร์ต่าง ๆ หรือ งานที่เกี่ยวข้องกับ machine learning เป็นต้น ซึ่งการทำงานของ Genetic Algorithm มีดังนี้



รูป 5 การทำงานของ genetic algorithm

ขั้นที่ 1 Initialization

การ Initialization คือการสร้างหรือกำหนด ประชากร(Population) รุ่นที่ 1 ขึ้นมาก่อน ซึ่งการสร้างประชากรนั้นก็อยู่ที่ปัญหาว่าจะออกแบบให้ประชากรนั้นเป็นแบบไหน (จะเรียกสมาชิกในประชากรว่าโครโมโซม (Chromosome) และ เรียกสมาชิกในโครโมโซม (Chromosome) เป็นสายอักขระบิทเลขฐานสองหรือเรียกว่าสายสตริง) โดยจะกำหนดแต่ละ โครโมโซมให้เป็นสายอักขระเลขฐานสองเช่น 1 0 1 0 1 0 เป็นต้น

ขั้นที่ 2 Selection

โดยจะใช้การคัดเลือกรูปแบบรูเล็ตต์ Roulette Wheel Selection จะต้องหาค่า Fitness หรือค่าความเหมาะสมของแต่ละประชากรและหาความน่าจะเป็นของแต่ละประชากรนำมาทำการสุ่มคัดเลือก ยิ่งความน่าจะเป็นของประชากรนั้นยิ่งมากโอกาสในการถูกคัดเลือกยิ่งมากขึ้นด้วย

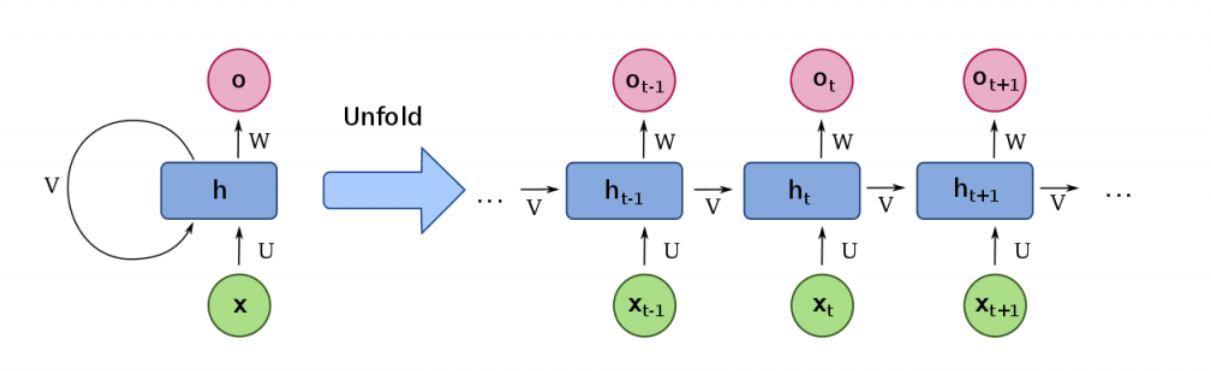
ขั้นที่ 3 Crossover

การ crossover หรือการไขว้เปลี่ยนนั้น จะเป็นกระบวนการในการสร้างลูกหลาน เกิดจากการสลับสายโครโมโซมเป็นคู่ลำดับ แต่จะต้องทำการหาคู่ลำดับก่อนโดยจะมีการกำหนดค่า crossover rate ระหว่าง 0 ถึง 1 เอาไว้เพื่อทำการเช็คกับค่าที่ถูกสุ่มขึ้นมาโดยจะสุ่มค่าระหว่าง 0 ถึง 1 เช่นกันถ้าเกิดค่าที่ถูกสุ่มขึ้นมามีค่าน้อยกว่า crossover rate ก็จะมีการนับเป็น 1 คู่ลำดับ จะไล่เช็คจนครบประชากรหารด้วยสอง

ขั้นที่ 4 Mutation

การกลายพันธุ์จะทำหลังการไขว้เปลี่ยน ก่อนที่จะทำการกลายพันธุ์ ต้องกำหนด Mutation rate โดยจะเป็นจำนวนเต็มบวกไม่เกิน 1 ซึ่งการกลายพันธุ์จะต้องสุ่มค่าตัวเลขระหว่าง 0 ถึง 1 เพื่อนำมาเช็คกับค่าของ Mutation rate ว่าอยู่ในเงื่อนไขหรือไม่ ถ้าตรงกับเงื่อนไขน้อยกว่าจะทำการสลับบิทจาก 0 เป็น 1 จาก 1 เป็น 0 โดยจะทำการไล่เช็คทุกบิทใน chromosome

3.6 Recurrent Neural Network (RNN) [8]



รูป 6 Recurrent Neural Network

Recurrent Neural Network (RNN) เป็นประเภทหนึ่งของโมเดลปัญญาประดิษฐ์ที่มีโครงสร้างแบบชั้นเดียวและสามารถใช้ในการประมวลผลข้อมูลลำดับหรือข้อมูลที่มีความเชื่อมโยงต่อกัน โดย RNN มีลักษณะพิเศษที่สามารถเก็บความจำเกี่ยวกับข้อมูลที่ผ่านมาในรูปแบบของ "สถานะภายใน" หรือ "memory" เพื่อใช้ในการประมวลผลข้อมูลในลำดับต่อไป

โครงสร้างพื้นฐานของ RNN ประกอบด้วยเซลล์ (cell) หรือโนดที่สามารถทำงานพิเศษในการประมวลผลข้อมูลลำดับ โดยแต่ละเซลล์จะมีส่วนประมวลผลที่หลากหลายตามการใช้งาน แต่ส่วนที่สำคัญคือส่วนที่เก็บข้อมูลหรือ "memory" ที่ช่วยให้ RNN สามารถจดจำสถานะก่อนหน้าและนำมาใช้ในการประมวลผลข้อมูลในลำดับถัดไป

การประยุกต์ใช้งานของ RNN อยู่ที่การจำแนกประเภทของข้อมูลที่เป็นลำดับ เช่น การแปลภาษา, การจดจำลำดับข้อความ, การทำนายชุดข้อมูลลำดับ (เช่น การทำนายราคาหุ้นในอนาคต), การแปลงเสียงเป็นข้อความ และอื่น ๆ โดยการใช้ RNN เข้ามาช่วยในการจดจำความสัมพันธ์ระหว่างข้อมูลที่เป็นลำดับจะช่วยเพิ่มประสิทธิภาพในการประมวลผลข้อมูลเหล่านี้ได้

บทที่ 4 ขั้นตอนและแผนการดำเนินงาน

ในบทนี้จะอธิบายถึงรายละเอียดในการทำงานในแต่ละขั้นตอน และแสดงแผนการดำเนินงานเพื่อวางแผนระยะเวลาที่ใช้ในการจัดทำโครงงานทั้งหมด

4.1 ขั้นตอนการดำเนินงาน

4.1.1 คิดหัวข้อและเสนอโครงงาน

- คิดหัวข้อโครงงานเพื่อเสนออาจารย์ที่ปรึกษาและเตรียมเสนอหัวข้อโครงงานต่อไป

4.1.2 รวบรวมข้อมูล

- เก็บรวบรวมข้อมูลที่เกี่ยวข้องกับโครงงาน ไม่ว่าจะเป็นการลงไปยังสถานที่จริงและการค้นหาผ่านอินเทอร์เน็ต

4.1.3 กำหนดขอบเขตโครงงานและวางแผนการดำเนินงาน

- กำหนดขอบเขตของโครงงาน เป้าหมายของโครงงาน วางแผนการดำเนินงาน

4.1.4 ศึกษางานที่เกี่ยวข้องกับโครงงาน

- ศึกษางานวิจัยที่เกี่ยวข้อง เพื่อนำมาปรับใช้ให้ตรงตามขอบเขตของโครงงาน

4.1.5 ศึกษาทฤษฎีและความรู้ที่เกี่ยวข้อง

- ศึกษาข้อมูลเกี่ยวกับ โมดูล minimalESN
- ศึกษาข้อมูลเกี่ยวกับโปรแกรม Visual Studio Code
- ศึกษาข้อมูลเกี่ยวกับภาษา Python
- ศึกษาข้อมูลเกี่ยวกับ numpy
- ศึกษาข้อมูลเกี่ยวกับ genetic algorithm

4.1.6 การออกแบบและพัฒนาโปรแกรม

- นำ genetic algorithm มาปรับใช้ร่วมกับ โมดูล minimalESN

4.1.7 ทดสอบโปรแกรม ปรับปรุง และประเมินผล

- ทดสอบดาต้าต่าง ๆ เพื่อตรวจสอบหาความแม่นยำของการประยุกต์ของ genetic algorithm
- ปรับปรุงและประเมินผลจากการทดสอบ

4.1.8 จัดทำเอกสารประกอบโครงงาน

- จัดทำเอกสารรายละเอียดของโครงงานตั้งแต่ขั้นตอนเริ่มต้นไปจนถึงขั้นตอนสุดท้ายของการทำโครงงาน

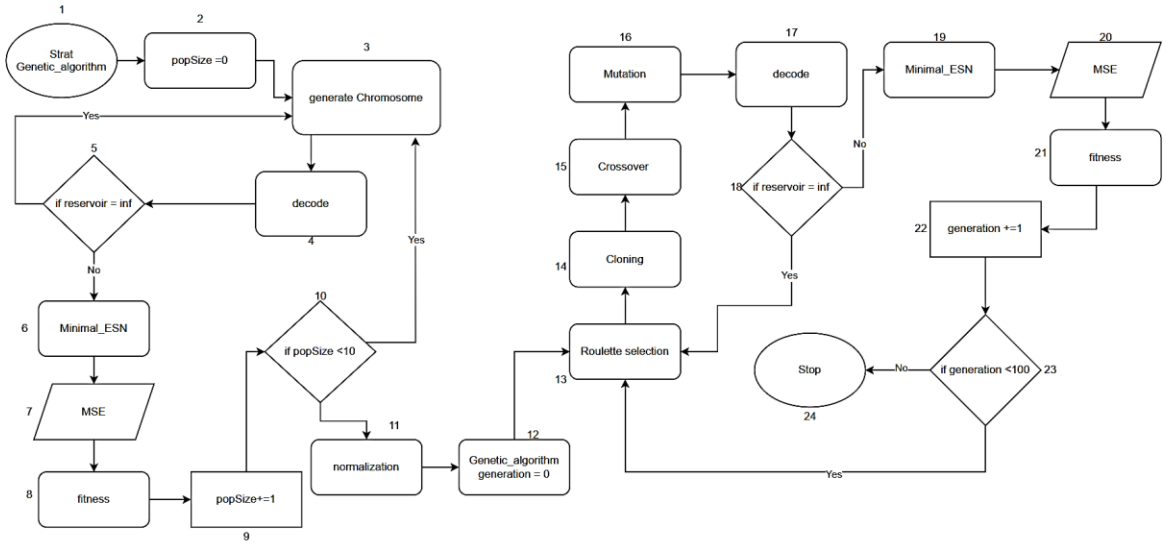
4.2 แผนการดำเนินงาน

ตารางที่ 1 แผนการดำเนินงาน

ขั้นตอนการดำเนินงาน	เดือน								
	ธ.ค. 2566	ม.ค. 2567	ก.พ. 2567	มี.ค. 2567	เม.ย. 2567	พ.ค. 2567	มิ.ย. 2567	ก.ค. 2567	ส.ค.-พ.ย. 2567
1. คิดหัวข้อและเสนอโครงงาน	↔								
2. รวมรวบข้อมูล	↔								
3. กำหนดขอบเขตโครงงาน และวางแผนการดำเนินงาน	↔								
4. ศึกษางานที่เกี่ยวข้องกับโครงงาน		↔	↔						
5. ศึกษาทฤษฎีและความรู้ที่เกี่ยวข้อง	↔	↔	↔						
6. การออกแบบและพัฒนาโปรแกรม			↔	↔	↔	↔	↔	↔	
7. ทดสอบโปรแกรม ปรับปรุงและประเมินผล			↔	↔	↔	↔	↔	↔	
8. จัดทำเอกสารประกอบโครงงาน	↔	↔	↔	↔	↔	↔	↔	↔	→

บทที่ 5 วิธีการดำเนินงาน

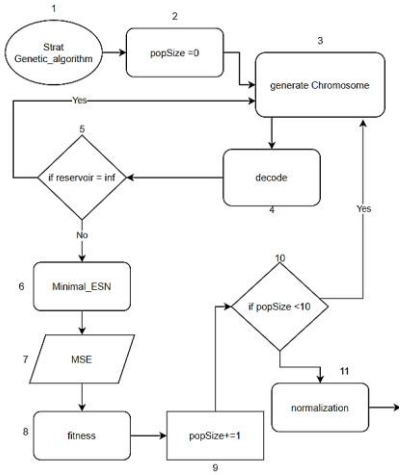
5.1 โครงสร้างหรือภาพรวมของโปรแกรม



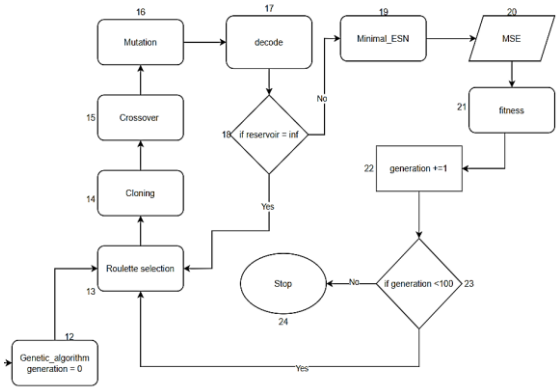
รูป 7 โครงสร้างการทำงานของโปรแกรม

5.2 ขั้นตอนการทำงานและโครงสร้างของโปรแกรม

โครงสร้างของโปรแกรมนั้นจะแบ่งการทำงานได้สองส่วนด้วยกัน ได้แก่ การทำงานก่อนที่จะเริ่ม generation และหลังเริ่ม generation



จะเป็นโครงสร้างก่อนเริ่ม generation



จะเป็นโครงสร้างหลังเริ่ม generation

ความแตกต่างระหว่างสองฝั่งนั้นจะมีดังนี้

ฝั่งก่อนเริ่ม generation

- จะโฟกัสไปที่เป็นการเริ่มต้นสร้างประชากร
- เป็นการ generate chromosome
- ทำจนครบ จำนวนประชากรที่ถูกกำหนดไว้

ฝั่งหลังทำ generation

- จะโฟกัสไปที่การวิวัฒนาการ
- โดยจะนำ chromosome ที่ถูก generate จนครบแล้วไปผ่านกระบวนการ roulette wheel selection crossover และ mutation
- ทำจนครบ generation ที่ถูกกำหนด

5.3 ขั้นตอนการทำงานของโปรแกรม

ขั้นตอนในการทำงานของโปรแกรมนั้นจะอ้างอิงจากรูปที่ 6 โครงสร้างในการทำงานของโปรแกรม

1. เริ่มต้นจากการกำหนด popSize และ clength และ generation จากไฟล์ .env เช่น

```
.env
popsize="50"
clength="10"
generations="100"
```

โดย popSize จะเป็นตัวกำหนดจำนวนของสายสตริง (ประชากร)

clength จะเป็นตัวกำหนดขนาดความยาวของสายสตริงแต่ละสาย

และ generation จะเป็นการกำหนดจำนวนของรุ่นที่ถูกส่งทอดสายสตริงไปยังรุ่นถัดไป

2. run โปรแกรมที่ genetic_algorithm.py
3. เริ่มต้นโดยการ generate Chromosome

```
34        for i in range(self.popsize):
35            self.chromosome()
```

3.1 โดยจำนวนครั้งที่จะต้องทำการ generate นั้น ขึ้นอยู่กับจำนวนของ popSize (สายสตริง) ที่กำหนด โดยบรรทัดที่ 34 จะเป็นลูปในการเรียกใช้ฟังก์ชันในการสร้างสายสตริงตามจำนวน popSize

```
100        def chromosome(self):
101            c = []
102            for j in range(self.clength):
103                if random.random() < 0.5:
104                    c.append(1)
105                else:
106                    c.append(0)
107            self.Chromosome.append(c)
108            #print(f"Chromosome {-1}: {c}")
109            self.decode(self.Chromosome)
```

3.2 ในฟังก์ชัน chromosome จะเป็นการสร้างสายอักขระเลขฐานสอง จะมีขนาดความยาวของสายเท่ากับ clength ในบรรทัดที่ 102 - 106 จะเป็นลูปในการสร้างสายสตริงโดยจะเป็นเลขบิท ระหว่าง 0 กับ 1 ซึ่งเลขบิทที่จะได้มานั้นเกิดจากการสุ่มค่าทศนิยมระหว่าง 0 ถึง 1 โดยจะเขียนเงื่อนไขเมื่อค่าที่ถูกสุ่มน้อยกว่า 0.5 ก็จะเก็บเป็นเลข 1 ถ้ามากกว่าเท่ากับ 0.5 ก็จะเก็บเป็นเลข 0 โดยท้ายที่สุดจะนำสาย chromosome ที่สร้างเสร็จหนึ่งสายเก็บไว้ในตัวแปรที่ชื่อ ว่า Chromosome

3.3 จากนั้นจะส่งตัวแปร Chromosome ไปยังฟังก์ชันที่ชื่อว่า decode

```
111        def decode(self,a):
112            #for i in range(len(a)):
113                a[-1].reverse()
114                get_c = 0
115                for j in range(self.clength):
116                    get_c += a[-1][j] * (2 ** j)
117                if get_c != 0 :
118                    self.x.append(get_c)
119                else :
120                    self.x.append(1)
121                a[-1].reverse()
122                if np.isin(self.x[-1], self.Error):
123                    print("ERROR == ",self.x[-1])
124                    print("ERRORRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRRR
125                    self.chromosome()
126                else:
127
128                    self.SendresSize()
```

4. ฟังก์ชัน decode นั้นจะเป็นการถอดรหัสบิทเลขฐานสองของแต่ละ Chromosome โดยตัวแปร Chromosome จะถูกส่งมายังฟังก์ชันนี้เก็บไว้ยังตัวแปร a

4.1 ในบรรทัดที่ 113 การถอดรหัสนั้นจะต้องมีการกลับสายบิทเสียก่อนเนื่องจากต้องทำการถอดจากขวาไปซ้าย

4.2 ในบรรทัดที่ 116 จะทำการถอดรหัสบิทเลขฐานสองให้กลายเป็นฐานสิบ

4.3 ในบรรทัดที่ 117 - 120 นั้นจะเป็นการเช็คเงื่อนไขว่าเลขฐานสิบของนั้นเป็นเลขศูนย์หรือไม่ ถ้าเกิดเป็นศูนย์จะเซ็ตค่าให้เป็นค่าที่น้อยที่สุดคือค่า 1

4.4 เมื่อเช็คแล้วว่าตรงเงื่อนไขไหนจะเก็บค่าเลขฐานสิบ ไว้ยังตัวแปร x

4.5 ในบรรทัดที่ 122 จะเป็นเงื่อนไขในการเช็ค ว่าในตัวแปร Error มีค่า x อยู่หรือไม่

ถ้าเป็นหนึ่งในตัวแปรที่ทำให้เกิดค่า inf จะทำการเรียกใช้ฟังก์ชัน chromosome ใหม่

ถ้าไม่ทำให้เกิดค่า inf ในบรรทัดที่ 126-128 ก็จะเรียกใช้ฟังก์ชัน SendresSize

4.5.1 ในตัวแปร Error นั้นคือค่าที่ทำให้เกิดค่า inf ในอนาคตซึ่งตัวแปร Error จะได้รับค่า data มาจาก ไฟล์ errorMSE.txt บรรทัดที่ 12

```
11        def __init__(self):
12            self.Error= np.loadtxt("errorMSE.txt")
```

4.5.2 ไฟล์ errorMSE.txt นั้นเกิดจากการรันค่า resSize (Reservoir size) ในไฟล์ minimalESN.py ซึ่งค่า resSize จะถูกรันตั้งแต่ 1 ถึง 1024 เพื่อเช็คหาค่าที่ทำให้ minimalESN.py เกิดค่า MSE ที่เป็น inf

4.5.3 MinimalESN [11] เป็นไฟล์ที่จะนำมาทำงานรวมกับ genetic algorithm(GA) โดยจะนำ GA นั้นไปปรับค่า resSize (Reservoir size) ในไฟล์ minimalESN โดยจะมีการปรับให้เป็น class minimalESN เพื่อให้อำนวยต่อการส่งค่าไปกลับระหว่าง GA และ minimalESN

5. SendresSize

```
69        def SendresSize(self):
70            self.meanF = 0
71            self.resSize = self.x[-1]
72            print("resSize = ",self.resSize)
73            breakCheck = 0
74            #for i in range(len(self.x)):
75            self.esn = minimalESN(self.x[-1])
76            self.mse.append(self.esn.M())
```

จะเป็นฟังก์ชันในการส่งค่า x หรือค่าเลขฐานสิบซึ่งเป็นตัวแทนของค่า resSize ใน minimalESN

5.1 ในบรรทัดที่ 75 จะเป็นการสร้าง object แล้วส่งค่า x เข้าไปยัง class minimalESN เพื่อนำค่า x นั้นไปใช้แทนค่า resSize

5.2 ใน class minimalESN จะแสดงผลค่า MSE ออกมา

5.3 ในบรรทัดที่ 76 จะทำการเก็บค่า MSE โดยจะเก็บใส่ตัวแปร mse ซึ่งเกิดจากการเรียกใช้ฟังก์ชัน M ใน class ของ minimalESN จะเก็บค่า MSE เข้ามา

```
96      def M(self):
97
98          return float(self.mse)
```

5.4 จะเรียกค่า MSE เหล่านี้ว่าค่า Fitness ซึ่งจะเป็นค่า ความเหมาะสม เนื่องจากเป็นค่า MSE ยิ่งค่าน้อยยิ่งดี เพราะฉะนั้นค่าความเหมาะสมจะต้องปรับตัวตามด้วย

6. MSE

หลังทำฟังก์ชัน SendresSize เสร็จแล้วจะเข้าสู่ฟังก์ชัน MSE เป็นฟังก์ชันที่หาค่าของ MSE และ resSize ที่ดีที่สุด และเก็บค่าผลลัพธ์ลงไฟล์ txt ชื่อ bestMSE.txt และ bestReservoir.txt

โดยบรรทัด 239 จะเป็นการสร้างไฟล์ bestMSE.txt และ บรรทัด 252 จะเป็นการเขียน mse ที่น้อยที่สุดลงไปในไฟล์

บรรทัด 257 จะเป็นการสร้างไฟล์ bestReservoir.txt และ 258 จะเป็นการเขียน resSize ลงไปในไฟล์

```
237     def MSE(self):
238         #self.loopcount= 1
239         with open('bestMSE.txt', 'a') as file:
240             self.smallest_mse = float('inf')
241             g = 0
242             c=0
243             for idx, i in enumerate(self.mse):
244                 if i != 0 and i < self.smallest_mse:
245                         self.smallest_mse = i
246                         g = idx
247             #print(g)
248             #self.getResSize()
249             self.GetR = self.x[g]
250             print("resSize == ",self.GetR)
251             #file.write(f"n = : {self.loopcount}\n")
252             file.write(f"{self.smallest_mse}\n")
253
254             #file.write(f"\n")
255             print("best mse ==== ",self.smallest_mse)
256
257         with open('bestReservoir.txt', 'a') as file:
258             file.write(f"{self.GetR}\n")
259         #self.loopcount+=1
260
```

6.1 โดยการเช็คนั้นจะหาค่า MSE ว่าค่าไหนน้อยที่สุดเนื่องจาก MSE ยิ่งค่าน้อยยิ่งดีจะต้องกำหนดตัวแปรที่มากที่สุดเพื่อนำมาเปรียบเทียบก่อน โดยในบรรทัดที่ 240 จะกำหนดตัวแปร smallest_mse ให้เป็นตัวเก็บค่า inf เพื่อที่จะเป็นค่าที่มากที่สุด

6.2 ในบรรทัด 243-246 จะเป็นลูปในการหาค่า mse ที่น้อยที่สุดและ reservoir โดยการหานั้นจะนำเงื่อนไขน้อยกว่า มาเทียบกับ smallest_mse และแทนค่าที่น้อยกว่าไปเรื่อย ๆ จนเจอค่าที่น้อยสุดและนำค่าตำแหน่งที่ตรงกับค่าที่น้อยที่สุดไปแทนใน x เพื่อหาค่า resSize

7. normalization

```
46                                  self.normalization()
```

ในฟังก์ชันนี้จะมีการปรับค่า MSE หรือ fitness ให้เหมาะสมและหาความน่าจะเป็นเพื่อนำไปใช้กับ method ต่อไป ให้แต่ละ fitness แทนด้วย

$$f[i], \quad i = 0,1,2,3, \dots, popSize - 1$$

โดยสูตรในการปรับค่า MSE นั้นจะให้ผลรวมของ MSE แทนด้วย F ลบด้วย แต่ละ แต่ละ fitness และเก็บไว้ที่ ตัวแปร p[i]

$$p[i] = F - f[i], i = 0,1,2,3, \dots, popSize - 1 \qquad (12)$$

จากนั้นจะหาผลรวมของแต่ละ p[i] แทนด้วย SumF เพื่อนำไปหาความน่าจะเป็นจะได้

$$SumF = \sum_{i=0}^{popSize-1} p[i] \qquad (13)$$

ต่อมาจะหาความน่าจะเป็นของแต่ mse ที่แทนด้วย p[I] จะได้เป็น

$$p[i] = \frac{p[i]}{SumF}, i = 0,1,2,3, \dots, popSize - 1 \qquad (14)$$

ตัวอย่างโค้ดในการนำมาประยุกต์

```
155        def normalization(self) :
156            self.SumF = 0
157            self.p =[]
158            self.F = 0
159            self.F=sum(self.mse)
160            self.p = self.mse
161            print(len(self.p),"  ",len(self.mse))
162            for i in range(self.popsize):
163                self.p[i] = self.F-self.p[i]
164                self.SumF += self.p[i]
165                #print(self.p)
166            for i in range(self.popsize):
167                self.p[i] = self.p[i]/(self.SumF )
168                print("Pop === ",self.p[i])
```

7.1 จะเก็บผลรวมของ MSE ไว้ที่ตัวแปร F ในบรรทัดที่ 159 และ เก็บตัวแปร MSE ไว้ที่ P ก่อนในบรรทัดที่ 160 เพื่อนำมาใช้ในอนาคต

7.2 ในบรรทัด 162-164 จะเป็นการจัดค่า MSE ก่อนที่จะนำไปหาค่าความน่าจะเป็นของ MSE แต่ละตัว

7.2.1 เนื่องจากค่า MSE ยิ่งน้อยยิ่งดีและยิ่งน้อยยิ่งค่าความน่าจะเป็นที่ถูกเลือกก็จะมากขึ้นด้วย ใน method ถัดไป จึงมีการจัดค่า โดยให้ค่าผลรวมทั้งหมด ลบกับค่าของแต่ละ MSE ในบรรทัดที่ 163 และในบรรทัดที่ 164 จะหาผลรวมของ MSE ที่ถูกจัดค่าเสร็จ โดยจะไล่บวกสะสมจนครบ MSE โดยเก็บไว้ในตัวแปร SumF

7.3 ในบรรทัดที่ 166-167 จะเป็นการเซ็ตหาความน่าจะเป็นของแต่ละ MSE หรือแต่ละค่า Fitness โดยจะนำ แต่ละ MSE หารด้วย SumF ก็จะได้ความน่าจะเป็นของแต่ละ MSE โดยเก็บไว้ในชื่อตัวแปร p[i]

8. Roulette Wheel Selection

Roulette Wheel Selection หรือ การคัดเลือกแบบรูเล็ตต์ จะเป็นกระบวนการคัดเลือกจากการสุ่มโดยใช้ความน่าจะเป็นตัวแทนของค่าความเหมาะสมหรือค่า Fitness ที่เป็นตัวแทนของแต่ละประชากร

ซึ่งความน่าจะเป็นของแต่ละ Fitness ได้มาจากการทำ normalization ก่อนหน้าที่แทนด้วยตัวแปร p โดยก่อนการคัดเลือกนั้นจะมีการสุ่มตัวเลขระหว่าง 0 ถึง 1 แทนด้วย r จากนั้นจะนำมาเช็คของแต่ละช่วงของความน่าจะเป็นของ MSE จะได้สูตรเป็น

$$\sum_{i=0}^{k-1} p[i] \leq r < \sum_{i=0}^{k} p[i] \qquad (15)$$

โดย k เป็นจำนวนของประชากร

นำไปประยุกต์กับโค้ดดังนี้

```
173        def RouletteSelection(self):
174            sum = 0
175            self.r = random.uniform(0, 1)
176            for i in range(self.popsize):
177                sum += self.p[i]
178                if(self.r <= sum):
179                    #print(i)
180                    return i
```

8.1 บรรทัดที่ 175 ตัวแปร r จะเป็นตัวแทนของค่าที่ถูกสุ่มระหว่าง 0 ถึง 1

8.2 บรรทัดที่ 176 ถึง 180 จะเป็น ลูปในการเช็คว่าค่า r นั้นอยู่ตรงกับประชากรกลุ่มที่เท่าไร

8.3 โดยการเช็คจะหาจากการบวกสะสมของ ความน่าจะเป็นดังบรรทัดที่ 177 โดยเก็บไว้ที่ตัว sum

8.4 จะนำ r มาเช็คกับค่าของ sum ว่าน้อยกว่าหรือเท่ากับหรือไม่ดังบรรทัดที่ 178 จากนั้นถ้าอยู่ในเงื่อนไขน้อยกว่าหรือเท่ากับก็จะเก็บตำแหน่ง ของประชากรเอาไว้

ตัวอย่าง

ให้ r=0.5 และความน่าจะเป็นของแต่ละ p[i] โดย i = 0, 1, 2, 3 เท่ากับ p[0] = 0.2, p[1] = 0.1, p[2] = 0.4, p[3] = 0.3

โดยจะนำ r=0.5 มาเทียบกับการบวกสะสมของ p จะได้

i=0 r <= p [0] จะได้ 0.5 <= 0.2 หรือไม่ ไม่ตรงกับเงื่อนไข

เงื่อนไขถัดไป i=1 r <= p[0]+p[1] จะได้ 0.5 <= 0.3 หรือไม่ ไม่ตรงกับเงื่อนไข

เงื่อนไขถัดไป i=2 r <= p[0]+p[1]+p[2] จะได้ 0.5 <= 0.7 หรือไม่ ตรงเงื่อนไข เพราะฉะนั้นจะเก็บค่า i = 2 เข้ามา และทำวนสุ่มหาค่า r และเปรียบเทียบกับค่า p แบบนี้จนครบ popSize หรือประชากรที่กำหนด

8.5 ทำจนครบ popSize

```
for j in range(self.popsize):
    self.select.append(self.RouletteSelection())
```

โดยจะเก็บตำแหน่งทั้งหมดไว้ใน ตัวแปร select

9. cloning

ส่วนนี้จะเป็นการทำต่อหลังเก็บตำแหน่งของที่ถูกเลือกครบแล้ว

```
183         def clonning(self):
184             self.newclone = []
185             for i in range(self.popsize):
186                 self.newclone.append(copy.deepcopy(self.Chromosome[self.select[i]]))
187                 print(f"newClone {i}: {self.newclone[i]}")
188             self.select = []
```

9.1 ในบรรทัดที่ 186 จะเก็บ Chromosome สายอักขระบิทเลขฐานสองที่ถูก generate ไว้โดยจะนำตำแหน่งที่ทำการคัดเลือกในตัวแปร select มาแทนในตำแหน่งที่ต้องการใน Chromosome

9.2 จะเก็บ Chromosome ที่ถูกเลือกไว้ในตัวแปร newclone

10. crossover

การ crossover หรือการไขว้เปลี่ยนนั้น จะเป็นการสร้างลูกหลาน เกิดจากการสลับสายโครโมโซมเป็นคู่ลำดับซึ่งตำแหน่งก่อนที่จะสลับนั้นจะเก็บจากการสุ่มโดยจะสุ่มตั้งแต่เลข 1 ถึง clength -1 (จะทำการสลับจากขวาไปซ้าย)และก่อนที่จะหาจำนวนคู่ลำดับในการสลับนั้นจะต้องมีการกำหนดค่า crossover rate หรือ โอกาสในการเกิดไขว้เปลี่ยน ซึ่งจะกำหนดไว้ที่ 0.8

โดยนำมาประยุกต์กับโค้ดดังนี้

```
194         def crossover(self):
195             self.crossoverRate = 0.8
196             self.ran = []
197             self.Cchrom = []
198             self.g = 0
199             #print("CP == ",self.CP)
200             for i in range(self.popsize // 2):
201                 self.ran.append(random.uniform(0, 1))
202                 if self.ran[-1] <= self.crossoverRate:
203                     self.g += 1
204             #print("G == ",self.g)
205             for i in range(0, self.g*2, 2):
206                 self.CP = random.randint(1, self.clength-1)
207                 self.realCP = self.clength -self.CP
208                 self.Cchrom.append(self.newclone[i][:self.realCP] + self.newclone[i + 1][self.realCP:])
209                 self.Cchrom.append(self.newclone[i + 1][:self.realCP] + self.newclone[i][self.realCP:])
210             #for i in range(self.popsize -self.g*2 ):
```

10.1 บรรทัดที่ 195 จะกำหนดตัวแปร crossoverRate ไว้ที่ 0.8

บรรทัด 196 ran จะเป็นการเก็บตัวแปรในการสุ่ม

บรรทัด 197 Cchrom จะเป็นตัวแปรในการเก็บคู่ลำดับหลังทำการ crossover

บรรทัด 198 g จะเก็บจำนวนนับเป็นคู่ลำดับ

บรรทัด 200-203 นั้นจะเป็นลูปของการสุ่มคู่ดำดับในการไขว้เปลี่ยนโดยลูปนั้นจะเป็นลูปครึ่งนึงของ popSize เนื่องจากจะทำงานเป็นคู่ลำดับ โดยตัวแปร ran จะเก็บค่าการสุ่มระหว่าง 0 ถึง 1 จากนั้นจะนำไปเช็คกับ crossoverRate ว่าน้อยกว่าหรือเท่ากับหรือไม่ ถ้าน้อยกว่าตรงเงื่อนไขจะมีการบวกจะสมค่า g

บรรทัด 205-209 จะเป็นลูปในการไขว้เปลี่ยนโดยจะเป็นลูปที่ขยับที่ละ 2

บรรทัด 206 จะจะสุ่มตัวเลขตำแหน่งของการสลับสายอักขระเลขฐานสองโดยจะสุ่มทุก ๆ คู่ลำดับ จะสุ่มตัวเลขระหว่าง 1 ถึง clength-1 เก็บไว้ในตัวแปร CP

บรรทัด 207 เนื่องจากจะต้องทำการตัดสายบิทนับจากทางขวาไปซ้ายเพราะฉะนั้นจะปรับตำแหน่งโดยจะนำ clength ลบกับ CP เพื่อที่จะได้ตำแหน่งที่ถูกนับไปจากทางขวา และเก็บไว้ยังตัวแปร realCP

บรรทัด 208-209 จะเป็นการสลับสายบิทระหว่างคู่ลำดับ โดยวิธีการนั้นจะเก็บแค่ตำแหน่งที่ยังไม่ถูกตัดนับจากทางซ้ายไปขวา ซึ่งก็คือ 0 ถึง realCP จากนั้นจะเอาสายบิทจากอีกสายมาต่อเข้าด้วยกัน โดยจะเริ่มจาก realCP ถึง clength และเก็บทั้งคู่ลำดับไว้ในตัวแปร Cchrom

11. mutation

Mutation หรือการกลายพันธุ์จะเป็นการกลับบิทจาก 0 เป็น 1 จาก 1 เป็น 0 ซึ่งจะเป็นกระบวนการที่ทำต่อจาก crossover ซึ่งก่อนการ mutation นั้นจะมีการกำหนด mutation rate ซึ่งก็คือ 0.2 โดยก่อนที่บิทจะกลับนั้นจะต้องทำการสุ่มค่า ระหว่าง 0 ถึง 1 เพื่อนำมาเช็คกับค่า mutation rate ว่าน้อยกว่าหรือไม่ถ้าน้อยกว่านั้นจะมีการกลับบิท

ตัวอย่างโค้ด

```
217         def mutation(self):
218             self.rm = []
219             self.mutationRate = 0.2
220             for i in range(self.g*2):
221                 for j in range(self.clength):
222                     self.rm.append(random.uniform(0, 1))
223                     if self.rm[-1] < self.mutationRate:
224                         if self.Cchrom[i][self.clength-j-1] == 0:
225                             self.Cchrom[i][self.clength-j-1] = 1
226                         elif self.Cchrom[i][self.clength-j-1] == 1:
227                             self.Cchrom[i][self.clength-j-1] = 0
228
229                 print(f"CmutationRate {i }: {self.Cchrom[i]}")
230             print("")
231             self.Cchrom.extend(self.newclone[self.g*2:self.popsize])
232             #for i in range(self.popsize):
233                 #print(f"Cchrom {i }: {self.Cchrom[i]}")
234             #print("")
235             self.callmeMaybe = copy.deepcopy(self.Chromosome)
236             self.Chromosome = self.Cchrom
237             #for i in range(self.popsize):
238                 #print(f"Chromosome {i }: {self.Chromosome[i]}")
239             self.decodeMutation(self.Cchrom)
```

ในบรรทัด 219 จะเป็นการกำหนดค่า mutation rate ให้เท่ากับ 0.2 และเก็บไว้ยังตัวแปร mutationRate

ในบรรทัด 220-227 จะเป็นลูปสองชั้น โดยลูปชั้นในแรกบรรทัด 220 จะเป็น g*2 ลูปเนื่องจากการทำ crossover ก่อนหน้าได้ g มาเป็นจำนวนคู่ลำดับจึง คูณสองเข้าไปเพื่อทำให้เป็นการไล่เช็คตามลำดับจำนวน crossover ที่ถูกทำไปก่อนหน้า ส่วนลูปชั้นที่สองในบรรทัด 221 จะเป็นการไล่เช็คบิทของประชากรตำแหน่งที่ i โดยบิทจะไล่เช็คตั้งแต่ 0 ถึง clength-1

และในบรรทัดที่ 222 จะเป็นการสุ่มตัวเลขระหว่าง 0 ถึง 1 เก็บไว้ในตัวแปร rm เพื่อนำมาเช็คกับ ค่า mutationRate ในบรรทัดที่ 223 ว่าน้อยกว่าหรือไม่ถ้าน้อยกว่าก็จะเข้าเงื่อนไขในการกลับบิทโดยจะมีเงื่อนไขเช็คก่อนกลับบิทว่าบิทที่ตำแหน่งนั้นเป็นเลข 0 หรือ 1 จากนั้นจะทำการกลับบิท

เนื่องจากการการทำ crossover นั้นมีโอกาสที่จะได้ทำไม่ครบคู่ลำดับตามจำนวนทั้งหมดของประชากรเนื่องจากจะเป็นการสุ่ม เพราะฉะนั้นจะต้องทำการเก็บสายอักขระในส่วนที่ไม่ได้เกิดการ crossover mutation ด้วยและ ในบรรทัดที่ 231 จะมาทดแทนในส่วนนี้ โดยจะนำสายอักขระตั้งแต่ตัวที่ g*2 ถึง popSize มาเพิ่มลงในตัวแปร Cchrom เพื่อให้ครบ popSize หรือจำนวนประชากรที่กำหนด

ในบรรทัด 235 จะเป็นการ copy ประชากรก่อนหน้าที่เก็บไว้ใน Chromosome มาเก็บไว้ในตัวแปร callmeMaybe ทั้งหมดเพื่อสำรองเอาไว้ใช้ถ้าเกิดหลังทำการ decode หรือถอดรหัสบิทตัว Cchrom แล้วตรงกับเงื่อนไขที่ทำให้เกิดค่า Error หรือ inf ก็จะนำตัวแปร callmeMaybe นี้ไปใช้ได้

ในบรรทัด 239 จะเป็นการเรียกใช้ฟังก์ชัน decodeMutation และส่งตัวแปร Cchrom ที่เก็บตัวแปร Chromosome ที่ผ่านกระบวนการ Roulette Wheel Selection crossover และ mutation เพื่อนำไปใช้ในฟังก์ชัน decode ต่อ

12. decodeMutation

จะคล้ายกับฟังก์ชัน decode ปกติแต่สิ่งที่เพิ่มเข้าหลังการถอดรหัสบิทแล้วนั้นคือหลังการเช็คค่า resSize ถ้าเกิดเจอค่าที่ตรงกับค่าที่อยู่ในตัวแปร Error แล้วจะมีการเพิ่มการทำ Roulette Wheel Selection, cloning, crossover, mutation ใหม่ทั้งหมด ถ้าเกิดไม่เจอค่า Error จะทำการเรียกฟังก์ชัน SendresSizeMu เพื่อทำการส่งไปยังclass minimalESN ต่อไป

ตัวอย่างโค้ด

```
128         def decodeMutation(self,a):
129                 self.x = []
130                 for i in range(len(a)):
131                     a[i].reverse()
132                     get_c = 0
133                     for j in range(self.clength):
134                         get_c += a[i][j] * (2 ** j)
135                     if get_c != 0 :
136                         self.x.append(get_c)
137                     else :
138                         self.x.append(1)
139                     a[i].reverse()
140                 if np.any(np.isin(self.x, self.Error)):
141                     self.select= []
142                     print("ERROR == ",self.x)
143                     print("ERRORRRR
```

13. SendresSizeMu

จะเป็นฟังก์ชันในการส่งค่า resSize ไปยัง class minimalESN เพื่อที่ทำการเก็บค่า MSE หรือ fitness เข้ามาเพื่อเปรียบเทียบหาค่าที่ดีที่สุดซึ่งฟังก์ชันจะเหมือนกับฟังก์ชัน SendresSize ในขั้นตอนที่ 5 แบบปกติ แต่จะมีการเพิ่ม การเรียกฟังก์ชัน MSE ในนั้น

```
    def SendresSizeMu(self):
        self.meanF = 0
        self.mse = []
        for i in range(len(self.x)):
            self.resSize = self.x[i]
            print("resSize = ",self.resSize)
        breakCheck = 0
        for i in range(len(self.x)):
            self.esn = minimalESN(self.x[i])
            self.mse.append(self.esn.M())
        self.MSE()
```

13.1 ซึ่งหลังเก็บค่า mse ทั้งหมดแล้วจะเรียกใช้ฟังก์ชัน MSE เพื่อที่จะหาค่าที่ดีที่สุดมาเก็บไว้และเปรียบเทียบเหมือนขั้นตอนที่ 6

14. generation

Generation คือการส่งต่อเพื่อตกทอดไปยังรุ่นต่อรุ่นซึ่งจะเปรียบเหมือนการนำ ประชากรหลังการทำ mutation เสร็จแล้วมาทำ Roulette Wheel Selection, cloning, crossover, mutation มาส่งทอดในการทำต่อไปจนถึง generation ที่กำหนดเอาไว้

15. ผลลัพธ์

ในแต่ละ generation นั้นจะทำการเก็บค่า mse และ resSize (reservoir) ที่ดีสุดของแต่ละ generation และนำไปเปรียบเทียบกับ generation ก่อนหน้าว่าเจอตัวที่ดีที่สุดใน generation ที่เท่าไร จากนั้นจะนำ generation ทั้งหมดมาหาค่าในทางสถิติ

ตัวอย่าง การเก็บ generation ที่ 1 ถึง 20

```
103   the best MSE in generation 0 is 6.078733295399675e-07 and reservoir is 1003.0 which is the 0 generation
104   the best MSE in generation 1 is 6.078733295399675e-07 and reservoir is 1003.0 which is the 0 generation
105   the best MSE in generation 2 is 6.078733295399675e-07 and reservoir is 1003.0 which is the 0 generation
106   the best MSE in generation 3 is 6.078733295399675e-07 and reservoir is 1003.0 which is the 0 generation
107   the best MSE in generation 4 is 6.078733295399675e-07 and reservoir is 1003.0 which is the 0 generation
108   the best MSE in generation 5 is 6.078733295399675e-07 and reservoir is 1003.0 which is the 0 generation
109   the best MSE in generation 6 is 6.078733295399675e-07 and reservoir is 1003.0 which is the 0 generation
110   the best MSE in generation 7 is 6.078733295399675e-07 and reservoir is 1003.0 which is the 0 generation
111   the best MSE in generation 8 is 6.078733295399675e-07 and reservoir is 1003.0 which is the 0 generation
112   the best MSE in generation 9 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 9 generation
113   the best MSE in generation 10 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 9 generation
114   the best MSE in generation 11 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 9 generation
115   the best MSE in generation 12 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 9 generation
116   the best MSE in generation 13 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 9 generation
117   the best MSE in generation 14 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 9 generation
118   the best MSE in generation 15 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 9 generation
119   the best MSE in generation 16 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 9 generation
120   the best MSE in generation 17 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 9 generation
121   the best MSE in generation 18 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 9 generation
122   the best MSE in generation 19 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 9 generation
123   the best MSE in generation 20 is 1.940535301040881e-07 and reservoir is 744.0 which is the 20 generation
```

เนื่องใน บรรทัดที่ 103 จะเป็นการเก็บที่ generation ที่ 0 จึงเป็นค่า MSE ที่ดีที่สุด จากนั้นก็จะไล่เช็คกับ generation ถัดไปว่าเจอตัวที่ดีกว่าหรือไม่ถ้าไม่เจอก็จะเขียนตัวเดิมออกมา จะเห็นได้ว่า ในบรรทัด 112 จะเจอค่า MSE น้อยกว่าจึงเขียนแทนที่และเขียนบอก generation ที่เจอว่าเจอที่ generation ที่เท่าไร ก็จะทำการหาแบบนี้ไปจนถึง generation ที่ 100

5.4 ขั้นตอนในการทดลอง

ในการทดลองนั้นเราจะนำดาต้าเซ็ต 5 ตัวด้วยกันนำมาเปรียบเทียบได้แก่ mackeyGlass_t17.txt, mackeyGlass_t30.txt, HENON.DAT, ROSSLER.DAT และ LORENZ.DAT โดยดาต้าเซ็ตเหล่านี้ถูกเรียกว่า chaotic time series เป็นรูปแบบของข้อมูลในดาต้าที่ดูเหมือนจะเป็นการถูกสุ่มขึ้นมา แต่ที่จริงแล้วมีสมการทางคณิตศาสตร์มากำกับเอาไว้ ซึ่งถูกนำไปใช้ในงานวิจัย เช่น การทำนายเวลาในอนาคต เป็นต้น

mackeyGlass Time Series

Mackey-Glass Time Series เป็นชุดข้อมูลที่เกิดจากสมการเชิงอนุพันธ์แบบไม่เชิงเส้นที่มีความล่าช้า (nonlinear time-delay differential equation) ซึ่งถูกพัฒนาโดย Leon Glass และ Dr. Michael Mackey

สมการ Mackey-Glass มีรูปแบบดังนี้

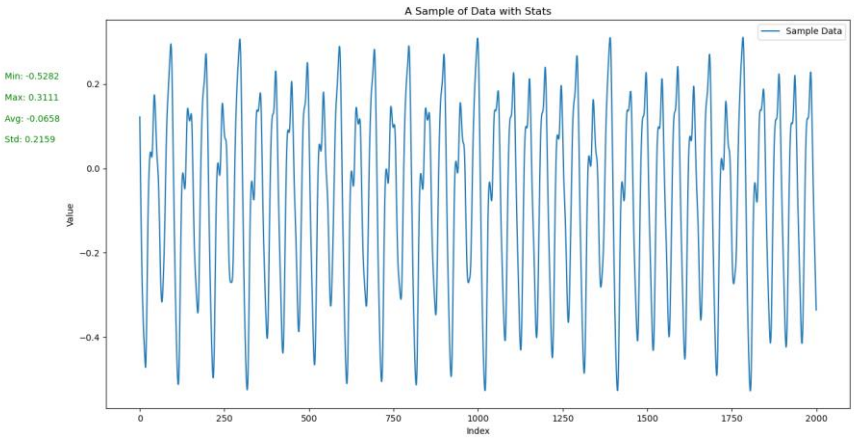
$$\frac{dx(t)}{dt} = \beta \frac{x(t-\tau)}{1+x(t-\tau)^n} - \gamma x(t) \qquad (16)$$

โดย

- $x(t)$ คือ ค่าของ x ณ เวลาที่ t
- γ คือ ค่าความหน่วงเวลา (time delay)

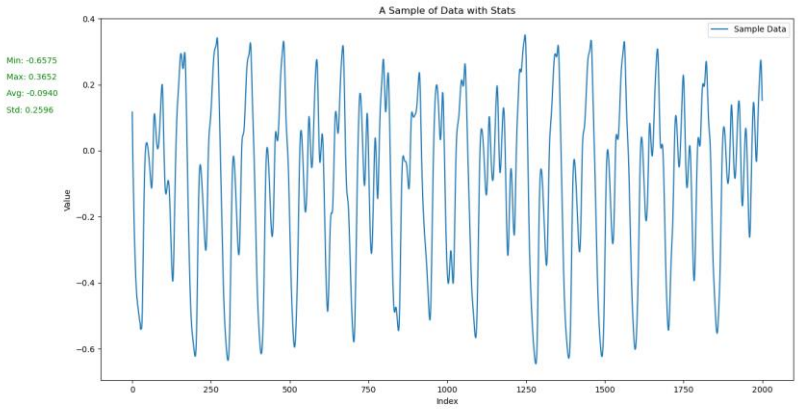
- β, γ, n คือค่าพารามิเตอร์ที่กำหนดพลวัตของระบบโดยจะต้องกำหนดด้วยตัวเอง

mackeyGlass_t17 คือ $\tau = 17$ (Time delay = 17) ตัวอย่างข้อมูลในรูปแบบกราฟ



รูป 8 กราฟแสดงตัวอย่างข้อมูลใน **mackeyGlass_t17.txt** ขนาด 2000 ตัว

mackeyGlass_t30 คือ $\tau = 30$ (Time delay = 30) ตัวอย่างข้อมูลในรูปแบบกราฟ



รูป 9 กราฟแสดงตัวอย่างข้อมูลใน **mackeyGlass_t30.txt** ขนาด 2000 ตัว

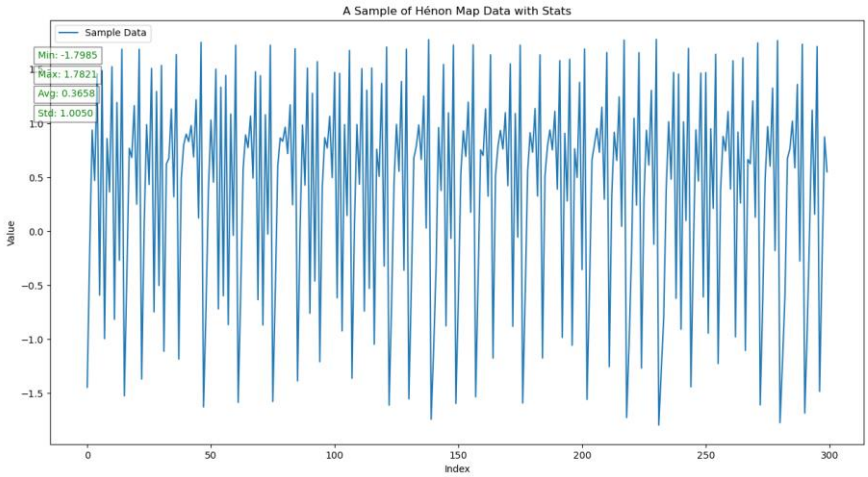
HENON.DAT

ข้อมูลดาต้าของ HENON นั้นถูกกำหนดด้วยสมการ

$$x' = a + b * y - x^2 \qquad (17)$$
$$y' = x \qquad (18)$$

โดยพารามิเตอร์ a และ b เป็นค่าคงที่ต้องกำหนดด้วยตัวเอง

ตัวอย่างข้อมูลดาต้ากำหนด a=1.4, b=0.3



รูป 10 กราฟแสดงตัวอย่างข้อมูลใน **HENON.DAT** ขนาด 300 ตัว

ROSSLER.DAT

ข้อมูลดาต้าของ HENON นั้นถูกกำหนดด้วยสมการ

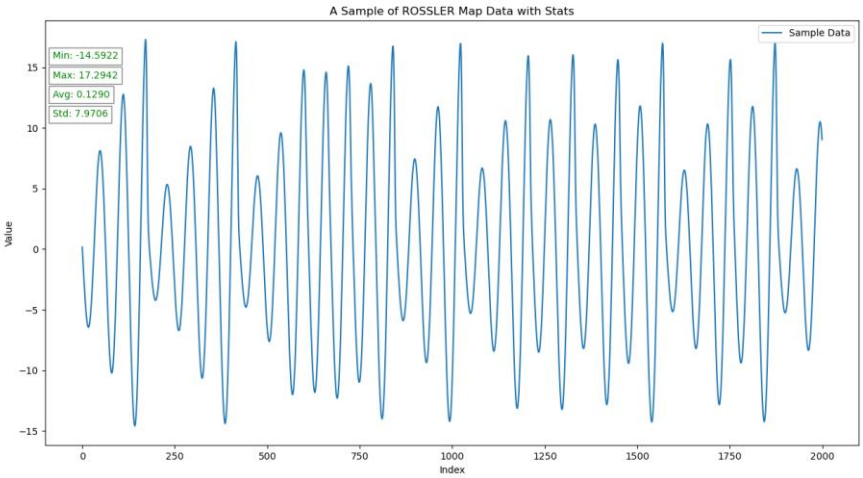
$$\frac{dx}{dt} = -z - y \qquad (19)$$

$$\frac{dy}{dt} = x + a * y \qquad (20)$$

$$\frac{dz}{dt} = b + z * (x - c) \qquad (21)$$

โดยพารามิเตอร์ a, b และ c เป็นค่าคงที่ต้องกำหนดด้วยตัวเอง

ตัวอย่างข้อมูลดาต้ากำหนด a=0.15, b=0.20, c=10.0



รูป 11 กราฟแสดงตัวอย่างข้อมูลใน **ROSSLER.DAT** ขนาด 2000 ตัว

LORENZ.DAT

ข้อมูลดาต้าของ LORENZ นั้นถูกกำหนดด้วยสมการ

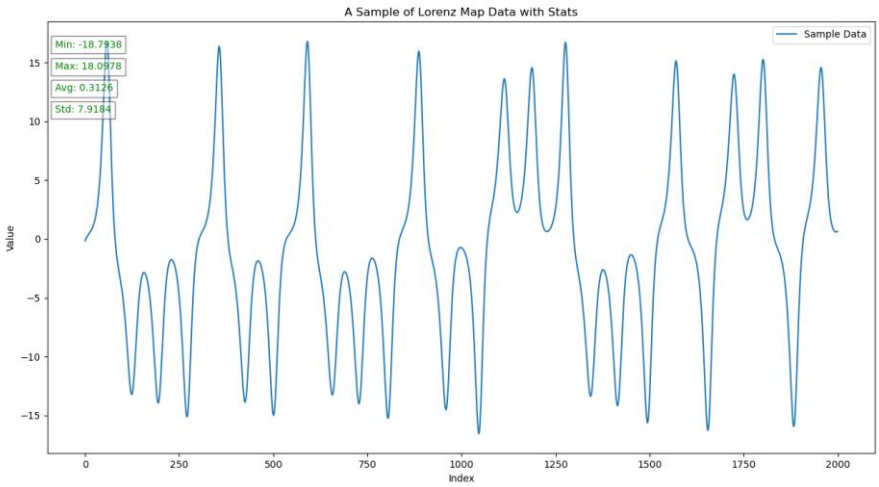
$$\frac{dx}{dt} = sigma * (y - x) \qquad (22)$$

$$\frac{dy}{dt} = r * x - y - x * z \qquad (23)$$

$$\frac{dz}{dt} = x * y - b * z \qquad (24)$$

โดยพารามิเตอร์ sigma, r และ b เป็นค่าคงที่ต้องกำหนดด้วยตัวเอง

ตัวอย่างข้อมูลดาต้ากำหนด sigma=10.0, r = 28.0, b = 8/3



รูป 12 กราฟแสดงตัวอย่างข้อมูลใน **LORENZ.DAT** ขนาด 2000 ตัว

ขอบเขตการทดลอง

กำหนดให้ขนาดของจำนวนของประชากร (popSize) ที่ต้องการทดสอบคือ 10, 50, 100 ความยาวของแต่ละสายสตริงหรือความยาวประชากร (Clength) คือ 10 และจำนวนรุ่น (Generation) = 100 โดยจะทำการรัน 30 ครั้ง

DATA	popSize	Clength	Generation	run
mackeyGlass_t17.txt	10, 50, 100	10	100	30
mackeyGlass_t30.txt	10, 50, 100	10	100	30
HENON.DAT	10, 50, 100	10	100	30
ROSSLER.DAT	10, 50, 100	10	100	30
LORENZ.DAT	10, 50, 100	10	100	30

โดยจะทำการเก็บค่า MSE ที่ดีที่สุดของแต่ละรุ่น และหาค่าเฉลี่ย ค่าที่มากที่สุด ค่าที่น้อยที่สุด ค่าส่วนเบี่ยงเบนมาตรฐาน และค่าขนาดของ reservoir ที่ทำให้เกิดค่า MSE ที่ดีที่สุดให้อยู่ในรูปแบบของตาราง

5.5 การวิเคราะห์ผลของการทดลอง

- นำค่า MSE ที่ดีสุดจากการหาขนาด reservoir ผ่าน genetic algorithm นั้นไปเปรียบเทียบกับ original ESN ที่ได้ทำการกำหนดขนาดของ reservoir ไว้ที่ 1000
- เก็บค่าสถิติของทุกดาต้าเซ็ตหลังทำการรัน 30 รัน

บทที่ 6 ผลการดำเนินงาน ข้อจำกัด และข้อเสนอแนะ

ในบทนี้จะกล่าวถึงผลการดำเนินงาน ข้อจำกัด และข้อเสนอแนะจากการทำโครงงานในเวลาที่ผ่านมา

6.1 ผลการดำเนินงาน

6.1.1 ดาต้าที่ใช้ทดสอบ

MackeyGlass_t17.txt

การเก็บผลลัพธ์

ผลลัพธ์จากการรันเก็บค่า MSE และ reservoir ที่ดีที่สุด ที่มีประชากร = 10 ความยาวสายบิท = 10 generation = 100 รัน 30 ครั้งด้วยกัน

ตัวอย่างการรันครั้งที่ 1

NO	MSE	RESERVOIR	TIME(SEC)
1	6.078733295399675e-07	1003	
2	9.320029942802152e-07	305	
3	8.544880937412949e-07	873	
4	9.320029942802152e-07	305	
5	1.1653602656841615e-06	711	
	...	...	
	...	...	
	...	...	
97	7.39969347226552e-07	804	
98	3.7547721341621534e-07	869	
99	1.7225304208073406e-06	887	
100	1.4858702317658086e-06	684	
101	3.697707653094072e-07	931	1365

จากตัวอย่างการเก็บผลครั้งที่ 1 ที่มี ประชากร = 10

ตัวอย่างการเก็บ 20 generation

```
the best MSE in generation 0 is 6.078733295399675e-07 and reservoir is 1003.0 which is the 0 generation
the best MSE in generation 1 is 6.078733295399675e-07 and reservoir is 1003.0 which is the 0 generation
the best MSE in generation 2 is 6.078733295399675e-07 and reservoir is 1003.0 which is the 0 generation
the best MSE in generation 3 is 6.078733295399675e-07 and reservoir is 1003.0 which is the 0 generation
the best MSE in generation 4 is 6.078733295399675e-07 and reservoir is 1003.0 which is the 0 generation
the best MSE in generation 5 is 6.078733295399675e-07 and reservoir is 1003.0 which is the 0 generation
the best MSE in generation 6 is 6.078733295399675e-07 and reservoir is 1003.0 which is the 0 generation
the best MSE in generation 7 is 6.078733295399675e-07 and reservoir is 1003.0 which is the 0 generation
the best MSE in generation 8 is 6.078733295399675e-07 and reservoir is 1003.0 which is the 0 generation
the best MSE in generation 9 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 9 generation
the best MSE in generation 10 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 9 generation
the best MSE in generation 11 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 9 generation
the best MSE in generation 12 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 9 generation
the best MSE in generation 13 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 9 generation
the best MSE in generation 14 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 9 generation
the best MSE in generation 15 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 9 generation
the best MSE in generation 16 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 9 generation
the best MSE in generation 17 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 9 generation
the best MSE in generation 18 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 9 generation
the best MSE in generation 19 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 9 generation
the best MSE in generation 20 is 1.940535301040881e-07 and reservoir is 744.0 which is the 20 generation
```

จะเห็นได้ว่า generation ที่ 0 ถึง 8 นั้นตัวที่ดีที่สุดที่เจอยังอยู่ใน generation ที่ 0 แต่เมื่อเจอ generation ที่ 9 ที่เจอ ค่า MSE ที่ดีกว่าแล้วนั้น ก็จะแสดงผล ค่าที่ดีที่สุดและเก็บค่า generation ที่เจอนั้นก็คือ 9 จนกระทั่งเจออีกครั้งใน generation ที่ 20

ผลลัพธ์จากการรันเก็บค่า MSE และ reservoir ที่ดีที่สุด ที่มีประชากร = 50 ความยาวสายบิท = 10 generation = 100 รัน 30 ครั้งด้วยกัน

ตัวอย่างการรันครั้งที่ 1

NO	MSE	RESERVOIR	TIME(SEC)
1	4.471348036235187e-07	443	
2	8.034482739876577e-07	371	
3	1.940535301040881e-07	744	
4	3.461731808418293e-07	555	
5	3.461731808418293e-07	555	
	...	...	
	...	...	
	...	...	
97	3.7338524677901096e-07	497	
98	3.7338524677901096e-07	497	
99	9.601451922412144e-07	656	
100	4.844373063868004e-07	353	
101	8.618438676211766e-07	817	6130

จากตัวอย่างการเก็บผลครั้งที่ 1 ที่มี ประชากร = 50

ตัวอย่างการเก็บ 20 generation

```
the best MSE in generation 0 is 4.471348036235187e-07 and reservoir is 443.0 which is the 0 generation
the best MSE in generation 1 is 4.471348036235187e-07 and reservoir is 443.0 which is the 0 generation
the best MSE in generation 2 is 1.940535301040881e-07 and reservoir is 744.0 which is the 2 generation
the best MSE in generation 3 is 1.940535301040881e-07 and reservoir is 744.0 which is the 2 generation
the best MSE in generation 4 is 1.940535301040881e-07 and reservoir is 744.0 which is the 2 generation
the best MSE in generation 5 is 1.940535301040881e-07 and reservoir is 744.0 which is the 5 generation
the best MSE in generation 6 is 1.940535301040881e-07 and reservoir is 744.0 which is the 5 generation
the best MSE in generation 7 is 1.940535301040881e-07 and reservoir is 744.0 which is the 5 generation
the best MSE in generation 8 is 1.940535301040881e-07 and reservoir is 744.0 which is the 5 generation
the best MSE in generation 9 is 1.940535301040881e-07 and reservoir is 744.0 which is the 5 generation
the best MSE in generation 10 is 1.940535301040881e-07 and reservoir is 744.0 which is the 10 generation
the best MSE in generation 11 is 1.940535301040881e-07 and reservoir is 744.0 which is the 10 generation
the best MSE in generation 12 is 1.940535301040881e-07 and reservoir is 744.0 which is the 10 generation
the best MSE in generation 13 is 1.940535301040881e-07 and reservoir is 744.0 which is the 10 generation
the best MSE in generation 14 is 1.940535301040881e-07 and reservoir is 744.0 which is the 10 generation
the best MSE in generation 15 is 1.940535301040881e-07 and reservoir is 744.0 which is the 10 generation
the best MSE in generation 16 is 1.940535301040881e-07 and reservoir is 744.0 which is the 10 generation
the best MSE in generation 17 is 1.940535301040881e-07 and reservoir is 744.0 which is the 10 generation
the best MSE in generation 18 is 1.940535301040881e-07 and reservoir is 744.0 which is the 10 generation
the best MSE in generation 19 is 1.940535301040881e-07 and reservoir is 744.0 which is the 10 generation
the best MSE in generation 20 is 1.940535301040881e-07 and reservoir is 744.0 which is the 10 generation
```

จากการเก็บผลเริ่มต้น generation ที่ 0 จะเจอตัวที่ดีกว่าใน generation ที่ 2 และเจออีกทีใน generation ที่ 5 และ 10 ตามลำดับ

ผลลัพธ์จากการรันเก็บค่า MSE และ reservoir ที่ดีที่สุด ที่มีประชากร = 100 ความยาวสายบิท = 10 generation = 100 รัน 30 ครั้งด้วยกัน

ตัวอย่างการรันครั้งที่ 1

NO	MSE	RESERVOIR	TIME(SEC)
1	3.631355407401875e-07	755	
2	3.4630092847251506e-07	555	
3	5.527842739138075e-07	779	
4	7.037160350923307e-07	778	
5	5.004395212657533e-07	929	
	...	...	
	...	...	
	...	...	
97	3.3873457473089005e-07	749	
98	3.3873457473089005e-07	749	
99	5.65904398616112e-07	516	
100	3.631355407401875e-07	755	
101	4.1272468452989164e-07	992	10895

จากตัวอย่างการเก็บผลครั้งที่ 1 ที่มี ประชากร = 100

ตัวอย่างการเก็บ 20 generation

```
the best MSE in generation 0 is 3.631355407401875e-07 and reservoir is 755.0 which is the 0 generation
the best MSE in generation 1 is 3.4630092847251506e-07 and reservoir is 555.0 which is the 1 generation
the best MSE in generation 2 is 3.4630092847251506e-07 and reservoir is 555.0 which is the 1 generation
the best MSE in generation 3 is 3.4630092847251506e-07 and reservoir is 555.0 which is the 1 generation
the best MSE in generation 4 is 3.4630092847251506e-07 and reservoir is 555.0 which is the 1 generation
the best MSE in generation 5 is 3.4630092847251506e-07 and reservoir is 555.0 which is the 1 generation
the best MSE in generation 6 is 3.4630092847251506e-07 and reservoir is 555.0 which is the 1 generation
the best MSE in generation 7 is 3.4630092847251506e-07 and reservoir is 555.0 which is the 7 generation
the best MSE in generation 8 is 3.4630092847251506e-07 and reservoir is 555.0 which is the 7 generation
the best MSE in generation 9 is 3.3873457473089005e-07 and reservoir is 749.0 which is the 9 generation
the best MSE in generation 10 is 3.3873457473089005e-07 and reservoir is 749.0 which is the 9 generation
the best MSE in generation 11 is 3.3873457473089005e-07 and reservoir is 749.0 which is the 9 generation
the best MSE in generation 12 is 3.3873457473089005e-07 and reservoir is 749.0 which is the 9 generation
the best MSE in generation 13 is 3.3873457473089005e-07 and reservoir is 749.0 which is the 9 generation
the best MSE in generation 14 is 3.3873457473089005e-07 and reservoir is 749.0 which is the 9 generation
the best MSE in generation 15 is 3.3873457473089005e-07 and reservoir is 749.0 which is the 9 generation
the best MSE in generation 16 is 3.3873457473089005e-07 and reservoir is 749.0 which is the 9 generation
the best MSE in generation 17 is 3.3873457473089005e-07 and reservoir is 749.0 which is the 9 generation
the best MSE in generation 18 is 3.3873457473089005e-07 and reservoir is 749.0 which is the 9 generation
the best MSE in generation 19 is 3.3873457473089005e-07 and reservoir is 749.0 which is the 9 generation
the best MSE in generation 20 is 3.3873457473089005e-07 and reservoir is 749.0 which is the 9 generation
```

จากการเก็บผลเริ่มต้น generation ที่ 0 จะเจอตัวที่ดีกว่าใน generation ที่ 1 และเจออีกที่ใน generation ที่ 7 และ 9 ตามลำดับ

MackeyGlass_t30.txt

เนื่องด้วยดาต้า MackeyGlass_t30.txt ทางโปรแกรมได้ตรวจเจอค่า reservoir ที่ทำโมดูล minimalESN เกิดค่า inf โดยจำนวนเกิดนั้นมีประมาณ 35% เราจึงไม่ได้ใช้ดาต้านี้มาทดลอง

HENON.DAT

การเก็บผลลัพธ์

ผลลัพธ์จากการรันเก็บค่า MSE และ reservoir ที่ดีที่สุด ที่มีประชากร = 10 ความยาวสายบิท = 10 generation = 100 รัน 30 ครั้งด้วยกัน

ตัวอย่างการรันครั้งที่ 1

NO.	MSE	RESERVOIR	TIME(SEC)
1	1.7197609107993481	203	
2	1.9489003202885054	801	
3	1.8885847076197149	537	
4	1.7794457803974124	277	
5	1.8012238683917028	788	
	...	...	
	...	...	
	...	...	
97	1.780387163024611	298	
98	1.8313474015733595	611	
99	1.9094911083686295	867	
100	1.8118199024705874	321	
101	1.787908480921671	963	1384

จากตัวอย่างการเก็บผลครั้งที่ 1 ที่มี ประชากร = 100

ตัวอย่างการเก็บ 20 generation

```
the best MSE in generation 0 is 1.7197609107993481 and reservoir is 203.0 which is the 0 generation
the best MSE in generation 1 is 1.7197609107993481 and reservoir is 203.0 which is the 0 generation
the best MSE in generation 2 is 1.7197609107993481 and reservoir is 203.0 which is the 0 generation
the best MSE in generation 3 is 1.7197609107993481 and reservoir is 203.0 which is the 0 generation
the best MSE in generation 4 is 1.7197609107993481 and reservoir is 203.0 which is the 0 generation
the best MSE in generation 5 is 1.7197609107993481 and reservoir is 203.0 which is the 0 generation
the best MSE in generation 6 is 1.7033160587190472 and reservoir is 974.0 which is the 6 generation
the best MSE in generation 7 is 1.7033160587190472 and reservoir is 974.0 which is the 7 generation
the best MSE in generation 8 is 1.7033160587190472 and reservoir is 974.0 which is the 8 generation
the best MSE in generation 9 is 1.7033160587190472 and reservoir is 974.0 which is the 8 generation
the best MSE in generation 10 is 1.7033160587190472 and reservoir is 974.0 which is the 8 generation
the best MSE in generation 11 is 1.7033160587190472 and reservoir is 974.0 which is the 8 generation
the best MSE in generation 12 is 1.7033160587190472 and reservoir is 974.0 which is the 8 generation
the best MSE in generation 13 is 1.7033160587190472 and reservoir is 974.0 which is the 8 generation
the best MSE in generation 14 is 1.7033160587190472 and reservoir is 974.0 which is the 8 generation
the best MSE in generation 15 is 1.7033160587190472 and reservoir is 974.0 which is the 8 generation
the best MSE in generation 16 is 1.7033160587190472 and reservoir is 974.0 which is the 8 generation
the best MSE in generation 17 is 1.7033160587190472 and reservoir is 974.0 which is the 8 generation
the best MSE in generation 18 is 1.7033160587190472 and reservoir is 974.0 which is the 8 generation
the best MSE in generation 19 is 1.7033160587190472 and reservoir is 974.0 which is the 8 generation
the best MSE in generation 20 is 1.7033160587190472 and reservoir is 974.0 which is the 8 generation
```

จากการเก็บผลเริ่มต้น generation ที่ 0 จะเจอตัวที่ดีกว่าใน generation ที่ 6 และเจออีกที่ใน generation ที่ 7 และ 8 ตามลำดับ

ROSSLER.DAT

เนื่องด้วยดาต้า ROSSLER.DAT ทางโปรแกรมได้ตรวจเจอค่า reservoir ที่ทำโมดูล minimalESN เกิดค่า inf โดยจำนวนเกิดนั้นมีประมาณ 87% เราจึงไม่ได้ใช้ดาต้านี้มาทดลอง

LORENZ.DAT

การเก็บผลลัพธ์

ผลลัพธ์จากการรันเก็บค่า MSE และ reservoir ที่ดีที่สุด ที่มีประชากร = 10 ความยาวสายบิท = 10 generation = 100 รัน 30 ครั้งด้วยกัน

ตัวอย่างการรันครั้งที่ 1

NO.	MSE	RESERVOIR	TIME(SEC)
1	47.74737445690237	984	
2	9.32735306570355	595	
3	4.802579470378394	866	
4	9.320920377227033	522	
5	9.320920377227033	522	
	...	...	
	...	...	
	...	...	
97	1.4822324048888798	913	
98	8.316314067974279	464	
99	1.1737281799595922	660	
100	8.383462754696659	912	
101	8.95617951959486	784	1810

จากตัวอย่างการเก็บผลครั้งที่ 1 ที่มี ประชากร = 10

ตัวอย่างการเก็บ 20 generation

```
the best MSE in generation 0 is 47.74737445690237 and reservoir is 984.0 which is the 0 generation
the best MSE in generation 1 is 9.32735306570355 and reservoir is 595.0 which is the 1 generation
the best MSE in generation 2 is 4.802579470378394 and reservoir is 866.0 which is the 2 generation
the best MSE in generation 3 is 4.802579470378394 and reservoir is 866.0 which is the 2 generation
the best MSE in generation 4 is 4.802579470378394 and reservoir is 866.0 which is the 2 generation
the best MSE in generation 5 is 1.4822324048888798 and reservoir is 913.0 which is the 5 generation
the best MSE in generation 6 is 1.4822324048888798 and reservoir is 913.0 which is the 5 generation
the best MSE in generation 7 is 1.4822324048888798 and reservoir is 913.0 which is the 5 generation
the best MSE in generation 8 is 1.4822324048888798 and reservoir is 913.0 which is the 5 generation
the best MSE in generation 9 is 1.4822324048888798 and reservoir is 913.0 which is the 5 generation
the best MSE in generation 10 is 1.4822324048888798 and reservoir is 913.0 which is the 5 generation
the best MSE in generation 11 is 1.1737281799595922 and reservoir is 660.0 which is the 11 generation
the best MSE in generation 12 is 1.1737281799595922 and reservoir is 660.0 which is the 11 generation
the best MSE in generation 13 is 1.1737281799595922 and reservoir is 660.0 which is the 11 generation
the best MSE in generation 14 is 0.8441497274371793 and reservoir is 894.0 which is the 14 generation
the best MSE in generation 15 is 0.8441497274371793 and reservoir is 894.0 which is the 14 generation
the best MSE in generation 16 is 0.8441497274371793 and reservoir is 894.0 which is the 14 generation
the best MSE in generation 17 is 0.8441497274371793 and reservoir is 894.0 which is the 14 generation
the best MSE in generation 18 is 0.8441497274371793 and reservoir is 894.0 which is the 14 generation
the best MSE in generation 19 is 0.8441497274371793 and reservoir is 894.0 which is the 14 generation
the best MSE in generation 20 is 0.8441497274371793 and reservoir is 894.0 which is the 14 generation
```

จากการเก็บผลเริ่มต้น generation ที่ 0 จะเจอตัวที่ดีกว่าใน generation ที่ 1 และเจออีกทีใน generation ที่ 2, 5, 11, 14 ตามลำดับ

6.1.2 การเก็บผลแบบตาราง

การเก็บผลนั้นจะเป็นค่าสถิติโดยมี ค่าเฉลี่ย MSE, ส่วนเบี่ยงเบนมาตรฐาน MSE, ค่าที่มากที่สุด (MAX), ค่าที่น้อยที่สุด(MIN) และจำนวนของ unit ใน reservoir ที่ให้ค่า MSE ดีที่สุด แบ่งแต่ละจำนวนขนาดของประชากร 10, 50 และ100 ที่ถูกเก็บค่า MSE ที่ดีที่สุดมา 30 รัน โดยแต่ละรันจะมีจำนวนรุ่น 100 รุ่นด้วยกัน

mackeyGlass_t17.txt

ตารางสถิติหาค่า เฉลี่ย MSE ส่วนเบี่ยงเบนมาตรฐาน max min ของประชากร 10, 50, 100

ประชากร	จำนวนรุ่น	ค่าเฉลี่ยMSE	ส่วนเบี่ยงเบนมาตรฐาน MSE	MAX	MIN	N
10	100	2.3475325972239933e-07	6.7763865134953e-08	3.697707653094072e-07	1.940535301040881e-07	744
50	100	1.988820249738532e-07	2.6002240644088797e-08	3.389083761970427e-07	1.940535301040881e-07	744
100	100	1.940764637017905e-07	0	1.940764637017905e-07	1.940764637017905e-07	744

N หมายถึงจำนวนของ unit ใน reservoir ที่ให้ค่า MSE ดีที่สุด

HENON.DAT

ประชากร	จำนวนรุ่น	ค่าเฉลี่ยMSE	ส่วนเบี่ยงเบนมาตรฐาน MSE	MAX	MIN	N
10	100	1.0342460921175856	0.005251178813058524	1.0432233940221771	1.0310806479056287	1
50	100	1.0342460921175856	0	1.0342460921175856	1.0342460921175856	1
100	100	1.0310806479056287	0	1.0310806479056287	1.0310806479056287	1

N หมายถึงจำนวนของ unit ใน reservoir ที่ให้ค่า MSE ดีที่สุด

LORENZ.DAT

ประชากร	จำนวนรุ่น	ค่าเฉลี่ยMSE	ส่วนเบี่ยงเบนมาตรฐาน MSE	MAX	MIN	N
10	100	0.28931212964738157	0.28396616292062843	1.1737281799595922	0.09713890231303939	671
50	100	0.09713890231303941	0	0.09713890231303941	0.09713890231303941	671
100	100	0.12972178324096487	0	0.12972178324096487	0.12972178324096487	671

N หมายถึงจำนวนของ unit ใน reservoir ที่ให้ค่า MSE ดีที่สุด

6.1.3 สรุปผลการทดลอง

การใช้ ESN ประยุกต์กับสิ่งต่าง ๆ ในปัจจุบันการประยุกต์ใช้ ESN นั้นได้ถูกนำไปใช้ในหลากหลายสาขา เช่น การทำนายอนุกรมเวลา การวิเคราะห์ข้อมูล การประมวลผลภาพเป็นต้น โดยงานวิจัยนี้ Application of the Genetic Algorithm in Echo State Networks เป็นการประยุกต์เพิ่มประสิทธิภาพของ ESN โดยเฉพาะในการเลือกขนาดของ reservoir ที่เหมาะสมที่สุด ซึ่งมีบทบาทสำคัญต่อความแม่นยำในการพยากรณ์หรือทำนายโดยเราสามารถสรุปผลการทดลองได้ดังนี้

- การที่จะเจอได้ค่า MSE ที่ดีที่สุดและเร็วที่สุดผ่าน genetic algorithm นั้น เราจะต้องใช้จำนวนประชากรที่มากขึ้นเพื่อทำการค้นหาค่า MSE

ตัวอย่าง การหาค่าที่ดีที่สุด ของดาต้า mackeyGlass_t17.txt ระหว่าง ประชากร 10, 50, 100 ตามลำดับ โดยค่า reservoir =744 จะได้ MSE= 1.940535301040881e-07 ซึ่งเป็นค่า MSE ที่ดีที่สุดในขอบเขตของ algorithm ที่หาได้ ซึ่งประชากรยิ่งมากยิ่งจะทำให้ได้เจอค่า MSE ได้ใน generation ที่ต้น ๆ ของการรัน

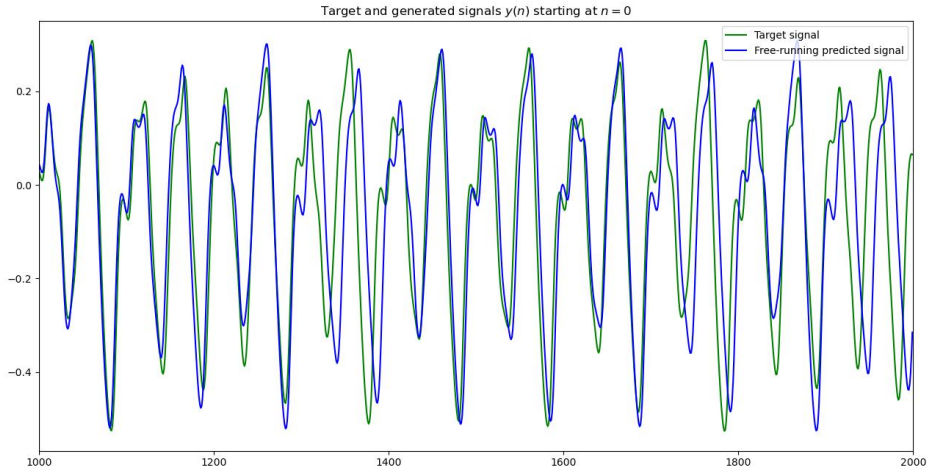
- การทดลองที่ได้รันมาทั้งหมด สรุปได้ว่าการหาค่า MSE ที่ดีที่สุดจะช่วยให้การทำนาย

มีความแม่นยำมากขึ้น

- ช่วยเพิ่มประสิทธิภาพให้การทำนาย ของโมดูล minimalESN

ผลการแสดงการทดลองเปรียบเทียบระหว่าง original ESN กับ GA-ESN กับดาต้าเซ็ต mackeyGlass_t17.txt โดยจะกำหนดตำแหน่งที่ 1000 ถึง 2000 ในโค้ดเพื่อให้กราฟแสดงออกมาเข้าใจได้ง่ายขึ้น

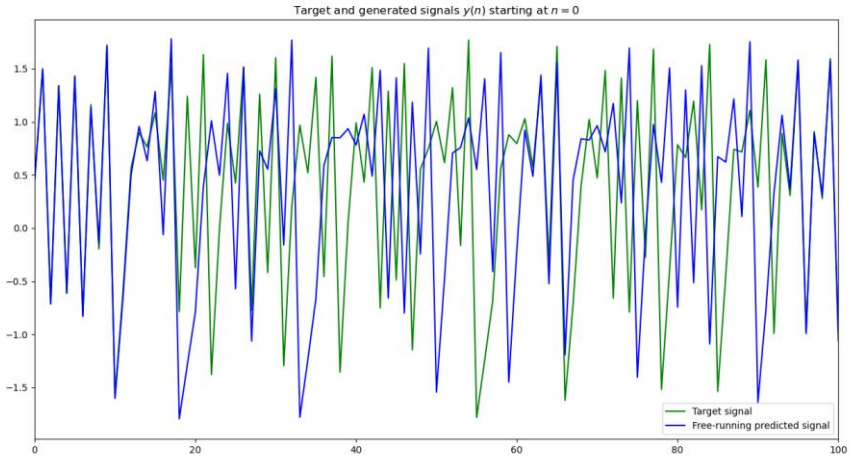
รูป 13 กราฟของ original ESN, reservoir size = 1000



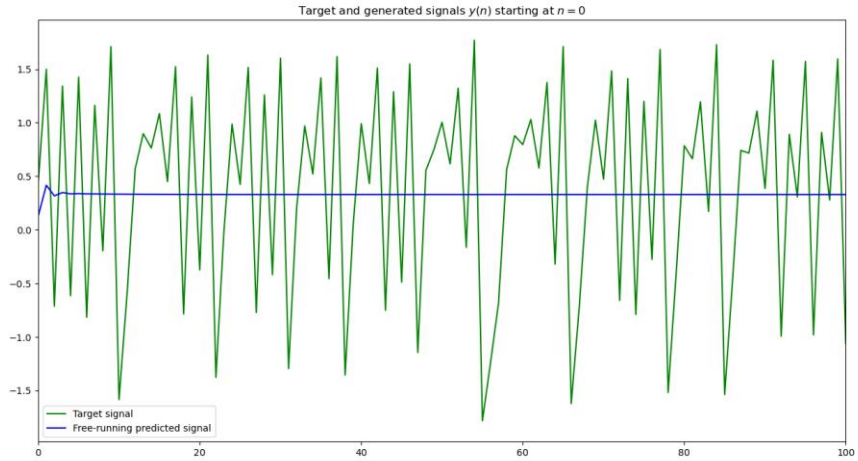
รูป 14 กราฟของ GA-ENS, reservoir size = 744

- เห็นได้ว่ารูปที่ 13 จะเป็นการทดลองแบบไม่ผ่านกระบวนการ genetic algorithm โดยผู้เขียน minimalESN ได้กำหนด reservoir ไว้ที่ 1000
- รูปที่ 14 จะเป็นการทดลองที่ผ่านกระบวนการ genetic algorithm โดยจะได้ค่า reservoir ที่ดีที่สุดคือ 744
- โดยการเปรียบเทียบจะเห็นได้ว่ารูปที่ 13 การทำนายของ original ESN นั้นมีความคลาดเคลื่อนหลังจากตำแหน่งที่ 1400 เป็นต้นไปอย่างมากถ้าเทียบกับรูปที่ 14 GA-ESN

แสดงการทดลองเปรียบเทียบระหว่าง original-ESN กับ GA-ESN กับดาต้าเซ็ต HENON.DAT โดยจะกำหนดตำแหน่งที่ 0 ถึง 100 ในโค้ดเพื่อให้กราฟแสดงออกมาเข้าใจได้ง่ายขึ้น



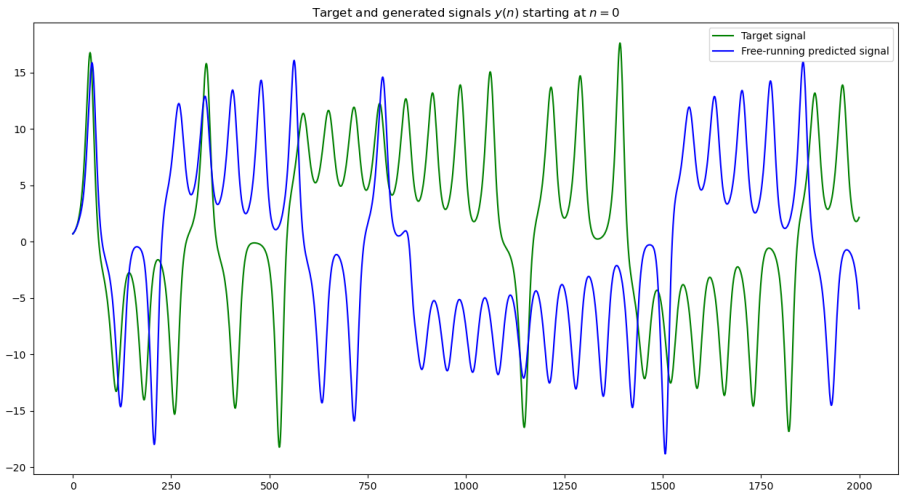
รูป 15 กราฟของ GA-ESN ,reservoir size= 1000



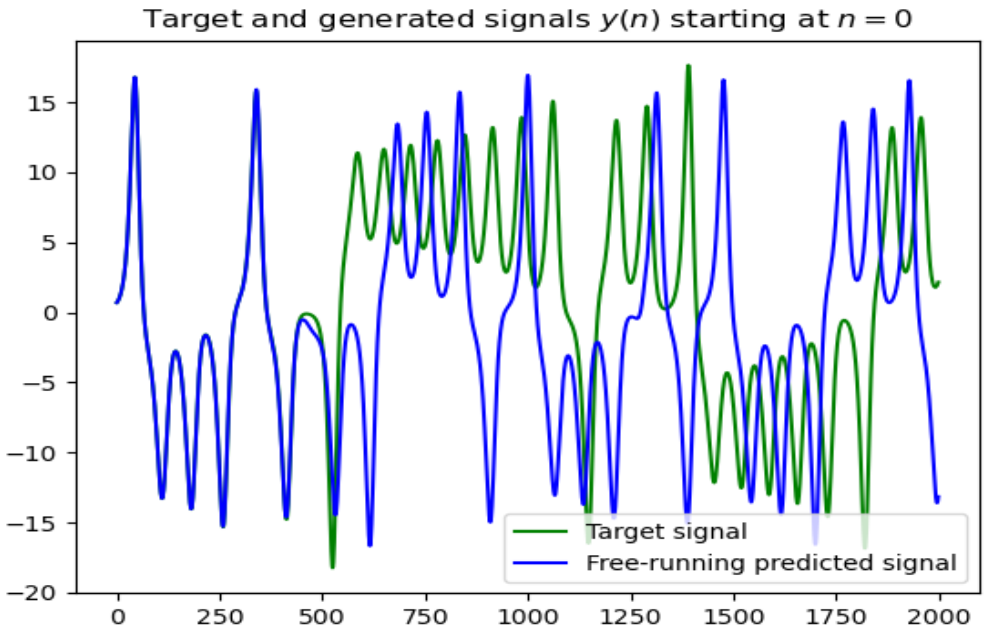
รูป 16 original ESN ,reservoir size= 1

โดยสามารถเห็นได้ว่าขนาดของ reservoir = 1 ทำให้เกิดปัญหาในการทำนายค่า original ESN ที่กำหนดขนาด reservoir = 1000 จึงมีกราฟที่ดีกว่า

แสดงการทดลองเปรียบเทียบระหว่าง original-ESN กับ GA-ESN กับดาต้าเซ็ต LORENZ.DAT โดยจะกำหนดตำแหน่งที่ 0 ถึง 100 ในโค้ดเพื่อให้กราฟแสดงออกมาเข้าใจได้ง่ายขึ้น



รูป 17 กราฟ original ESN, reservoir size = 1000



รูป 18 กราฟ GA-ESN, reservoir = 671

- จะเห็นได้ว่าในรูปที่ 17 กราฟในการทำนายจาก original ESN ที่กำหนดขนาดของ reservoir = 1000 มานั้น ไม่ได้เป็นไปตามที่เป้าหมายที่ต้องการเป็นอย่างมาก เมื่อนำไปเปรียบเทียบกับกับรูปที่ 18 นั้นการทำนายมีแนวโน้มที่ใกล้เคียงกับเป้าหมายมากกว่าเป็นอย่างมาก

6.2 ข้อจำกัด

ข้อจำกัดที่พบเจอมีดังนี้

- ข้อจำกัดของโมดูล minimalESN ไม่สามารถแก้ไข randomseed ได้จำกัดค่าที่ใช้ randomseed แค่ 42 เท่านั้น
- ดาต้าไฟล์ MackeyGlass_t30.txt พบค่า reservoir ที่ทำให้เกิดค่า inf มากเกินไป พบเจอโดยประมาณ 35% ของขอบเขตในการ generate chromosome
- ดาต้าไฟล์ ROSSLER.DAT พบค่า reservoir ที่ทำให้เกิดค่า inf มากเกินไป พบเจอโดยประมาณ 86% ของขอบเขตในการ generate chromosome
- ในดาต้าของ HENON.DAT จากการใช้ genetic algorithm มาหาค่าขนาดของ reservoir โดยให้ผลลัพธ์ค่า MSE ออกมาได้ดีที่สุดซึ่งได้ค่าขนาดของ reservoir = 1 มาจึงเกิดปัญหาในการทำนายกราฟ

6.3 ข้อเสนอแนะ

- หากต้องการเพิ่มประสิทธิภาพของขอบเขตการการทำนาย สามารถเพิ่มจำนวนขนาดความยาวของประชากรได้หรือเรียกอีกแบบนึงว่าขนาดความยาวของสายบิท

บรรณานุกรม

[1] Ahmad, Zohaib and Nie, Kaizhe and Qiao, Junfei and Yang, Cuili, (2563). Hybrid Structure Based PSO for ESN Optimization,[ออนไลน์]. เข้าถึงเมื่อ 14 พฤศจิกายน 2567. เข้าถึงได้จาก https://ieeexplore.ieee.org/document/8958625

[2] Zhu, Yingqin and Zhang, Zhaozhao and Wang, Qiuwan, (2564). Optimization of echo state network behavior space based on microbial genetic algorithm. [ออนไลน์]. เข้าถึงเมื่อ 14 พฤศจิกายน 2567. เข้าถึงได้จาก https://ieeexplore.ieee.org/abstract/document/9602533

[3] Mantas Lukoševičius , minimalESN("A practical guide to applying echo state networks "), [ออนไลน์]. เข้าถึงเมื่อ 14 พฤศจิกายน 2567. เข้าถึงได้จาก https://mantas.info/code/simple_esn/

[4] Python, Wikipedia, [ออนไลน์]. เข้าถึงเมื่อ 14 พฤศจิกายน 2567. เข้าถึงได้จาก https://en.wikipedia.org/wiki/Python_(programming_language)

[5] Visual Studio Code, Wikipedia, [ออนไลน์]. เข้าถึงเมื่อ 14 พฤศจิกายน 2567. เข้าถึงได้จาก https://en.wikipedia.org/wiki/Visual_Studio_Code

[6] numpy, Wikipedia, [ออนไลน์]. เข้าถึงเมื่อ 14 พฤศจิกายน 2567. เข้าถึงได้จาก https://en.wikipedia.org/wiki/NumPy

[7] Krittamet Kotawong, (2564). genetic algorithm, [ออนไลน์]. เข้าถึงเมื่อ 14 พฤศจิกายน 2567. เข้าถึงได้จาก https://krittamet-project.medium.com/genetic-algorithms-%E0%B8%84%E0%B8%B7%E0%B8%AD%E0%B8%AD%E0%B8%B0%E0%B9%84%E0%B8%A3-%E0%B8%9E%E0%B8%A3%E0%B9%89%E0%B8%AD%E0%B8%A1%E0 %B8%95%E0%B8%B%E0%B8%A7%E0%B8%AD%E0%B8%A2%E0%B9%88%E0%B8%B2%E0%B8%87%E0%B9%81%E0%B8%A5%E0%B8%B0%E0%B9%82%E0%B8%84%E0%B9%89%E0%B8%94-57c41fbac251

[8] Surapong Kanoktipsatharporn, Recurrent Neural Network (RNN), 2019-12-12 [ออนไลน์]. เข้าถึงเมื่อ 14 พฤศจิกายน 2567. เข้าถึงได้จาก https://www.bualabs.com/archives/3103/what-is-rnn-recurrent-neural-network-what-is-gru-gated-recurrent-unit-teach-how-to-build-rnn-gru-with-python-nlp-ep-9/

[9] MinimalESN.py [ออนไลน์]. เข้าถึงเมื่อ 14 พฤศจิกายน 2567. เข้าถึงได้จาก https://mantas.info/wp/wp-content/uploads/simple_esn/minimalESN.py

ภาคผนวก ก การทำงานของ Genetic Algorithm

minimal_esn.py

การทำงานของโมดูล minimal_esn มีหน้าที่แค่ปรับดาต้าเซ็ตที่ต้องการใช้ในการทดสอบโดยสามารถเซ็ตได้ดังนี้

```
17        data = np.loadtxt('MackeyGlass_t17.txt')
```

และในส่วนของขนาด reservoir

```
26        resSize = resZ
```

เราได้ทำการเซ็ตไว้ที่ตัวแปร resZ เป็นตัวแปรสำหรับในการรับค่าขนาดของ reservoir เข้ามาจาก genetic algorithm

Genetic_algorithm.py

โดยจะเป็นไฟล์หลักที่ใช้ในการรันการทำงานของ genetic algorithm กับฟังก์ชันต่าง ๆ ก่อนที่จะเก็บผลลัพธ์ให้แก่ผู้ใช้งานนั้น มีขั้นตอนดังนี้

1. จำนวนประชากร ขนาดของประชากร และ รุ่นที่ต้องการ

```
.env
popsize="10"
clength="10"
generations="10"
```

รูปผนวก ที่ ก 1 ไฟล์ .env

ในไฟล์นี้จะเป็นการกำหนดจำนวนประชากรแทนด้วย popsize ขนาดของประชากรแทนด้วย clenth และรุ่น แทนด้วย generations

2. เรียกใช้ class genetic algorithm

```
def main():
        #for i in range(10):
            start = time.perf_counter()
            GA = GenaticAlgorithm()
            end = time.perf_counter()
            print("time : ",end - start)
            sec = int(end - start)
            with open('Time.txt', 'a') as file:
                file.write(f"{sec}\n")
```

รูปผนวก ที่ ก 2 ฟังก์ชัน main

โดยฟังก์ชันนี้เป็น ฟังก์ชัน main จะทำการเรียกใช้ class genetic algorithm และมีการนับเวลาเป็นหน่วยวินาที ต่อการรันหนึ่งรัน และจะเขียนเก็บไว้ที่ไฟล์ Time.txt

3.ฟังก์ชันเริ่มต้น init

```
    def __init__(self):
        self.Error= np.loadtxt("errorMSE.txt")
        self.Error = self.Error.astype(int)
        self.popsize()
        self.length()
        self.gc = os.getenv("generations")
        self.generation_count =int(self.gc)
        self.run_generations(self.generation_count)
```

รูปผนวก ที่ ก 3 ฟังก์ชัน __init__(self)

ฟังก์ชัน __init__(self) จะเป็นการเก็บไฟล์ errorMSE.txt มาเก็บไว้ในตัวแปร self.Error ซึ่งจะเป็นตัวแปรในการเช็คหาค่าที่จะทำให้เกิดความผิดพลาดในฟังก์ชันถัด ๆ ไป และมีการเรียกใช้ฟังก์ชันอีกสามฟังก์ชันด้วยกันคือ popsize(), length() และ run_generation(self.generation_count) โดยตัวแปร self.gc ที่เกิดจากการโหลดไฟล์ .env ของตัวแปร “generations” มาเก็บค่าไว้จากนั้น convert เป็น int และถูกเก็บไว้ใน self.generation_count

4. ฟังก์ชัน popsize และ length

```
21        def popsize(self):
22            self.ps = os.getenv("popsize")
23            self.popsize = int(self.ps)
24
25        def length(self):
26            self.c= os.getenv("clength")
27            self.clength=int(self.c)
```

รูปผนวก ที่ ก 4 ฟังก์ชัน popsize(self) และ ฟังก์ชัน length(self)

สองฟังก์ชันนี้จะเป็นฟังก์ชันเก็บค่าจากไฟล์ .env โดยตัวแปร self.popsize จะเก็บผ่านตัวแปร “popsize” และ self.clength จะเก็บผ่านตัวแปร “clength” จากไฟล์ .env เช่นกัน

5. ฟังก์ชัน run_generation ส่วนที่ 1

```
29        def run_generations(self,t):
30            self.Chromosome = []
31
32            self.mse = []
33            self.x = []
34            for i in range(self.popsize):
35                self.chromosome()
36            for i in range(self.popsize):
37                print(f"Chromosome {i}: {self.Chromosome[i]}")
38            #print(self.mse)
39
40            self.MSE()
41            gen=0
42            smallest_Gen  = float('inf')
```

รูปผนวก ที่ ก 5 ฟังก์ชัน run_generation(self,t) 1

โดยในส่วนที่ 1 นั้นจะเป็นการสร้าง โครโมโซมโดยจะทำการเรียกใช้ ฟังก์ชัน chromosome() โดยจะทำการเรียกจนครบ popsize ที่เรากำหนด

6. ฟังก์ชัน chromosome

```
82
83          def chromosome(self):
84              c = []
85              for j in range(self.clength):
86                  if random.random() < 0.5:
87                      c.append(1)
88                  else:
89                      c.append(0)
90              self.Chromosome.append(c)
91              #print(f"Chromosome {-1}: {c}")
92              self.decode(self.Chromosome)
```

รูปผนวก ที่ ก 6 ฟังก์ชัน chromosome(self)

ในฟังก์ชันนี้จะเป็นการสร้างสายโครโมโซม โดยจะเป็นสายอักขระเลขฐานสองจากการทำการสุ่มตัวเลข 0 หรือ 1 ตามจำนวน clength มาเก็บเอาไว้ในตัวแปร Chromosome จนครบ จากนั้นจะส่งค่า Chromosome ไปยัง ฟังก์ชัน decode

```
Chromosome 0: [0, 1, 1, 1, 1, 0, 0, 0, 1, 1]
Chromosome 1: [1, 0, 0, 1, 1, 0, 0, 0, 1, 0]
Chromosome 2: [0, 1, 1, 0, 0, 1, 0, 1, 1, 0]
Chromosome 3: [1, 0, 0, 1, 0, 0, 1, 1, 1, 1]
Chromosome 4: [0, 1, 0, 1, 1, 1, 1, 0, 0, 1]
Chromosome 5: [0, 1, 1, 0, 0, 1, 0, 1, 0, 0]
Chromosome 6: [0, 1, 1, 1, 0, 1, 0, 0, 0, 1]
Chromosome 7: [1, 0, 0, 0, 1, 1, 0, 1, 1, 0]
Chromosome 8: [1, 1, 0, 1, 1, 0, 1, 1, 1, 1]
Chromosome 9: [1, 0, 0, 0, 0, 0, 1, 1, 1, 0]
```

ตัวอย่างการเก็บ Chromosome popsize = 10, clength = 10

7. ฟังก์ชัน decode

```
def decode(self,a):
    #for i in range(len(a)):
        a[-1].reverse()
        get_c = 0
        for j in range(self.clength):
            get_c += a[-1][j] * (2 ** j)
        if get_c != 0 :
            self.x.append(get_c)
        else :
            self.x.append(1)
        a[-1].reverse()
        if np.isin(self.x[-1], self.Error):
            print("ERROR == ",self.x[-1])
            print("ERRORRRR
```

9. ฟังก์ชัน MSE

```
221                 self.decodeMutation(self.Cchrom)
222             def MSE(self):
223                 #self.loopcount= 1
224                 with open('bestMSE.txt', 'a') as file:
225                     self.smallest_mse = float('inf')
226                     g = 0
227                     c=0
228                     for idx, i in enumerate(self.mse):
229                         if i != 0 and i < self.smallest_mse:
230                                 self.smallest_mse = i
231                                 g = idx
232                     #print(g)ss
233                     #self.getResSize()
234                     self.GetR = self.x[g]
235                     print("resSize == ",self.GetR)
236                     #file.write(f"n = : {self.loopcount}\n")
237                     file.write(f"{self.smallest_mse}\n")
238
239                     #file.write(f"\n")
240                     print("best mse ==== ",self.smallest_mse)
241
242                 with open('bestReservoir.txt', 'a') as file:
243                     file.write(f"{self.GetR}\n")
244                 #self.loopcount+=1
```

รูปผนวก ที่ ก 9 ฟังก์ชัน MSE(self)

โดยฟังก์ชันนี้จะเป็นการเช็คค่า MSE หาค่าที่ดีสุดจากจำนวน MSE ที่มีทั้งหมด และเขียนค่าลงไปยังไฟล์ bestMSE.txt และเช็คหาค่า reservoir ที่ตรงกับค่าที่ทำให้เกิด MSE แล้วเขียนไปยังไฟล์ bestReservoir.txt

10. ฟังก์ชัน run_generation ส่วนที่ 2

```
43                 for i in range(t):
44
45                     self.select= []
46                     self.normalization()
47                     for j in range(self.popsize):
48                         self.select.append(self.RouletteSelection())
49                     print("select",self.select)
50                     self.clonning()
51                     self.crossover()
52                     self.mutation()
53
54                     #print(t)
55                     print("generation  === ", i )
```

รูปผนวก ที่ ก 10 ฟังก์ชัน run_generation(self,t) 2

ในส่วนต่อมาของฟังก์ชัน run_generation จะเป็นส่วนของการทำ normalization cloning crossover และ mutation

11. ฟังก์ชัน normalization

```
140        def normalization(self) :
141            self.SumF = 0
142            self.p =[]
143            self.F = 0
144            self.F=sum(self.mse)
145            self.p = self.mse
146            print(len(self.p),"  ",len(self.mse))
147            for i in range(self.popsize):
148                self.p[i] = self.F-self.p[i]
149                self.SumF += self.p[i]
150                #print(self.p)
151            for i in range(self.popsize):
152                self.p[i] = self.p[i]/(self.SumF )
153                print("Pop === ",self.p[i])
```

รูปผนวก ที่ ก 11 ฟังก์ชัน normalization(self)

เป็นฟังก์ชันในการ แปลงค่าแต่ละ MSE มาเป็นความน่าจะเป็นเพื่อนำไปใช้ต่อในฟังก์ชันถัดไป เนื่องด้วย ค่า MSE ยิ่งน้อยยิ่งดี เราจึงต้องมีการ normalize ก่อนเพื่อหาความน่าจะเป็นโดยจะ นำผลรวมของ MSE ทั้งหมดไปลบกับ ค่าแต่ละ MSE จากนั้น เราค่อยนำแต่ละค่าของ MSE มาหาความน่าจะเป็นของแต่ละตัว

12. ฟังก์ชัน RouletteSelection

```
158        def RouletteSelection(self):
159            sum = 0
160            self.r = random.uniform(0, 1)
161            for i in range(self.popsize):
162                sum += self.p[i]
163                if(self.r <= sum):
164                    #print(i)
165                    return i
166
```

รูปผนวก ที่ ก 12 ฟังก์ชัน RouletteSelection(self)

ฟังก์ชันนี้เป็นการนำความน่าจะเป็นของแต่ละ MSE มาคัดเลือกโดยใช้วิธีสุ่มค่าระหว่าง 0 ถึง 1 นำค่าที่สุ่มนั้นไปเช็คกับการบวกสะสมของความน่าจะเป็นตามลำดับของ MSE ถ้าเกิดค่าที่สุ่มออกมานั้นมีค่าน้อยกว่าหรือเท่ากับผลรวมเราก็จะเก็บลำดับของการบวกสะสมนั้น

13. ฟังก์ชัน cloning

```python
    def cloning(self):
        self.newclone = []
        for i in range(self.popsize):
            self.newclone.append(copy.deepcopy(self.Chromosome[self.select[i]]))
            print(f"newClone {i}: {self.newclone[i]}")
        self.select = []
```

รูปผนวก ที่ ก 13 ฟังก์ชัน cloning(self)

Cloning จะเป็นการนำตำแหน่งลำดับของ ฟังก์ชัน RouletteSelection ที่ผ่านกระบวนการคัดเลือกมาแล้วมาสร้าง Chromosome ใหม่โดยนำลำดับของตำแหน่งที่ถูกคัดเลือกมาแทนใน Chromosome

ตัวอย่าง

```
Chromosome 0: [0, 1, 1, 1, 1, 0, 0, 0, 1, 1]
Chromosome 1: [1, 0, 0, 1, 1, 0, 0, 0, 1, 0]
Chromosome 2: [0, 1, 1, 0, 0, 1, 0, 1, 1, 0]
Chromosome 3: [1, 0, 0, 1, 0, 0, 1, 1, 1, 1]
Chromosome 4: [0, 1, 0, 1, 1, 1, 1, 0, 0, 1]
Chromosome 5: [0, 1, 1, 0, 0, 1, 0, 1, 0, 0]
Chromosome 6: [0, 1, 1, 1, 0, 1, 0, 0, 0, 1]
Chromosome 7: [1, 0, 0, 0, 1, 1, 0, 1, 1, 0]
Chromosome 8: [1, 1, 0, 1, 1, 0, 1, 1, 1, 1]
Chromosome 9: [1, 0, 0, 0, 0, 0, 1, 1, 1, 0]
```

โครโมโซมเริ่มต้น

```
select [1, 4, 3, 2, 4, 9, 8, 5, 4, 9]
```

แต่ละตำแหน่งของการคัดเลือก

```
newClone 0: [1, 0, 0, 1, 1, 0, 0, 0, 1, 0]
newClone 1: [0, 1, 0, 1, 1, 1, 1, 0, 0, 1]
newClone 2: [1, 0, 0, 1, 0, 0, 1, 1, 1, 1]
newClone 3: [0, 1, 1, 0, 0, 1, 0, 1, 1, 0]
newClone 4: [0, 1, 0, 1, 1, 1, 1, 0, 0, 1]
newClone 5: [1, 0, 0, 0, 0, 0, 1, 1, 1, 0]
newClone 6: [1, 1, 0, 1, 1, 0, 1, 1, 1, 1]
newClone 7: [0, 1, 1, 0, 0, 1, 0, 1, 0, 0]
newClone 8: [0, 1, 0, 1, 1, 1, 1, 0, 0, 1]
newClone 9: [1, 0, 0, 0, 0, 0, 1, 1, 1, 0]
```

Cloning จะทำการสร้างผ่านแต่ละตำแหน่งของ select จะเห็นได้ว่า newClone ตัวที่ 0 จะเป็นตำแหน่งที่ 1 ของ Chromosome เป็นต้น

14. ฟังก์ชัน Crossover

```python
def crossover(self):
    self.crossoverRate = 0.8
    self.ran = []
    self.Cchrom = []
    self.g = 0
    #print("CP == ",self.CP)
    for i in range(self.popsize // 2):
        self.ran.append(random.uniform(0, 1))
        if self.ran[-1] <= self.crossoverRate:
            self.g += 1
    #print("G == ",self.g)
    for i in range(0, self.g*2, 2):
        self.CP = random.randint(1, self.clength-1)
        self.realCP = self.clength - self.CP
        self.Cchrom.append(self.newclone[i][:self.realCP] + self.newclone[i + 1][self.realCP:])
        self.Cchrom.append(self.newclone[i + 1][:self.realCP] + self.newclone[i][self.realCP:])
    #for i in range(self.popsize -self.g*2 ):

    for i in range(self.g*2):
        print(f"CrossoverC {i }: {self.Cchrom[i]}")
```

รูปผนวก ที่ ก 14 ฟังก์ชัน crossover(self)

ฟังก์ชันนี้จะเป็นกระบวนการไขว้เปลี่ยนหลังการทำ cloning โดยจะทำกันเป็นคู่ลำดับของ newclone โดยอัตราในการเกิดคู่ลำดับนั้นเกิดจากการกำหนดค่า crossoverRate ที่มากกว่า 0 และไม่เกิน 1 เพื่อนำมาเช็คกับการสุ่มค่าระหว่าง 0 ถึง 1 ถ้าค่าที่สุ่มมาน้อยกว่าหรือเท่ากับก็จะทำการนับเป็น 1 คู่ลำดับโดยเราจะทำการสุ่มจนครบจำนวนครึ่งนึงของ popsize หลังจากนั้นจะทำการไขว้เปลี่ยนโดยตำแหน่งที่เราจะไขว้เปลี่ยนนั้นจะเกิดจากการสุ่มโดยจะมีขอบเขตระหว่าง 1 ถึง clength-1(ไขว้เปลี่ยนบิทนับจากทางขวาไปซ้าย)

15. ฟังก์ชัน mutation

```python
def mutation(self):
    self.rm = []
    self.mutationRate = 0.2
    for i in range(self.g*2):
        for j in range(self.clength):
            self.rm.append(random.uniform(0, 1))
            if self.rm[-1] < self.mutationRate:
                if self.Cchrom[i][self.clength-j-1] == 0:
                    self.Cchrom[i][self.clength-j-1] = 1
                elif self.Cchrom[i][self.clength-j-1] == 1:
                    self.Cchrom[i][self.clength-j-1] = 0

        print(f"CmutationRate {i }: {self.Cchrom[i]}")
    print("")
    self.Cchrom.extend(self.newclone[self.g*2:self.popsize])
    #for i in range(self.popsize):
        #print(f"Cchrom {i }: {self.Cchrom[i]}")
    #print("")
    self.callmeMaybe = copy.deepcopy(self.Chromosome)
    self.Chromosome = self.Cchrom
    #for i in range(self.popsize):
        #print(f"Chromosome {i }: {self.Chromosome[i]}")
    self.decodeMutation(self.Cchrom)
```

รูปผนวก ที่ ก 15 ฟังก์ชัน mutation(self)

ในฟังก์ชันนี้จะเป็นการกลับบิทจาก 0 เป็น 1 จาก 1 เป็น 0 โดยจะทำต่อจากหลังทำคู่ลำดับของ crossover จนครบ อัตราในการกลับบิทนั้นเราจะกำหนดค่า mutationRate ระหว่างเลข 0 ถึง 1 และนำมาเช็คกับค่าที่ถูกสุ่มขึ้นมาระหว่าง 0 ถึง 1 ถ้าน้อยกว่าค่า mutationRate ก็จะมีการกลับบิทโดยจะทำการไล่เช็คทุกตัวใน Chromosome หลังเสร็จกระบวนการนี้ก็จะมีการส่ง Chromosome ทั้งหมดไปยังฟังก์ชัน decodeMutation

16. ฟังก์ชัน decodeMutation

```
def decodeMutation(self,a):
        self.x = []
        for i in range(len(a)):
            a[i].reverse()
            get_c = 0
            for j in range(self.clength):
                get_c += a[i][j] * (2 ** j)
            if get_c != 0 :
                self.x.append(get_c)
            else :
                self.x.append(1)
            a[i].reverse()
        if np.any(np.isin(self.x, self.Error)):
            self.select= []
            print("ERROR == ",self.x)
            print("ERRORRRR
```

17. ฟังก์ชัน SendresSizeMu

```
def SendresSizeMu(self):
    self.meanF = 0
    self.mse = []
    for i in range(len(self.x)):
        self.resSize = self.x[i]
        print("resSize = ",self.resSize)
    breakCheck = 0
    for i in range(len(self.x)):
        self.esn = minimalESN(self.x[i])
        self.mse.append(self.esn.M())
    self.MSE()
```

รูปผนวก ที่ ก 17 ฟังก์ชัน SendresSizeMu

ในฟังก์ชันนี้จะเหมือนกับฟังก์ชัน SendresSize ปกติคือจะส่งค่า reservoir ไปยัง class minimalESN และรับค่าเข้ามาและมีการเรียกฟังก์ชัน MSE โดยจะเช็คหาค่า MSE ที่ดีสุดพร้อมกับค่า reservoir ที่ตรงกัน

18. ผลลัพธ์

ผลลัพธ์นั้นเราจะได้มาสามไฟล์ด้วยกันคือ ไฟล์ bestMSE.txt bestReservoir.txt และ Time.txt เราสามารถเรียกใช้ไฟล์ Checkgen.py เพื่อหาค่าที่ดีที่สุดโดยเริ่มจาก generation ที่ 0 ไปจนถึงที่เรากำหนดโดยจะไล่เช็คว่า จะเจอค่า MSE ที่ดีกว่า generation เริ่มต้นตัวที่เท่าไรและจะเจอใน generation ถัด ๆ ไปใน generation ไหน

ตัวอย่าง

```
the best MSE in generation 0 is 8.244999184233323e-07 and reservoir is 765.0 which is the 0 generation
the best MSE in generation 1 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 1 generation
the best MSE in generation 2 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 2 generation
the best MSE in generation 3 is 4.4376049665200514e-07 and reservoir is 918.0 which is the 2 generation
the best MSE in generation 4 is 1.940535301040881e-07 and reservoir is 744.0 which is the 4 generation
the best MSE in generation 5 is 1.940535301040881e-07 and reservoir is 744.0 which is the 5 generation
the best MSE in generation 6 is 1.940535301040881e-07 and reservoir is 744.0 which is the 5 generation
the best MSE in generation 7 is 1.940535301040881e-07 and reservoir is 744.0 which is the 5 generation
the best MSE in generation 8 is 1.940535301040881e-07 and reservoir is 744.0 which is the 5 generation
the best MSE in generation 9 is 1.940535301040881e-07 and reservoir is 744.0 which is the 5 generation
```

เป็นต้น

GAforNoError.py

จะเหมือนกันไฟล์ genetic_algorithm.py เกือบทุกอย่างแต่ไม่มีการเช็คสำหรับ Error โดยเราจะใช้ไฟล์นี้ในการใช้ร่วมกับดาต้าเซ็ต HENON.DAT และ LORENZ.DAT

Checkgen.py

จะเป็นไฟล์สำหรับหลังการหา generation จนครบที่กำหนดและจำนวนครั้งที่รันไป 30 ครั้ง

```python
import numpy as np
bestInGeneration = np.loadtxt('bestMSE.txt')
bestInGenerationRE = np.loadtxt('bestReservoir.txt')
N=30
with open('bestGeneration.txt', 'a') as file:

    smallest_Gen = float('inf')  # Initialize with infinity
    gen = 0  # Initialize generation index
    g=0
    c=0
    v=0
    for j in range(N):

        for i in range(g,g+101):
            if bestInGeneration[i] <= smallest_Gen:
                smallest_Gen = bestInGeneration[i]
                v=c
            file.write(f"the best MSE in generation {gen} is {smallest_Gen} and reservoir is {bestInGenerationRE[v]} which is the {v-g} generation \
            gen+=1
            c+=1
        gen = 0
        file.write("\n")

        g+=101
        smallest_Gen = float('inf')

```

รูปผนวก ที่ ก 18 Checkgen.py

ถ้าจะเช็คในการรันแค่ 1 ครั้งตามและรัน generation จนครับที่กำหนดแล้วสามารถปรับค่าได้ที่ N

```python
N=30
```

ปรับ N เป็น 1 ได้

โดยจะเป็นการเช็คหาในแต่ละรอบใน generation ว่าค่า MSE ที่ดีที่สุดจะอยู่ที่ generation ที่เท่าไร reservoir อะไร

ตัวอย่าง

```
the best MSE in generation 0 is 3.7338524677901096e-07 and reservoir is 497.0 which is the 0 generation
the best MSE in generation 1 is 3.7338524677901096e-07 and reservoir is 497.0 which is the 1 generation
the best MSE in generation 2 is 3.7338524677901096e-07 and reservoir is 497.0 which is the 1 generation
the best MSE in generation 3 is 3.7338524677901096e-07 and reservoir is 497.0 which is the 1 generation
the best MSE in generation 4 is 3.7338524677901096e-07 and reservoir is 497.0 which is the 1 generation
the best MSE in generation 5 is 3.7338524677901096e-07 and reservoir is 497.0 which is the 1 generation
the best MSE in generation 6 is 3.7338524677901096e-07 and reservoir is 497.0 which is the 1 generation
the best MSE in generation 7 is 3.7338524677901096e-07 and reservoir is 497.0 which is the 1 generation
the best MSE in generation 8 is 3.7338524677901096e-07 and reservoir is 497.0 which is the 1 generation
the best MSE in generation 9 is 3.7338524677901096e-07 and reservoir is 497.0 which is the 1 generation
the best MSE in generation 10 is 3.7338524677901096e-07 and reservoir is 497.0 which is the 1 generation
```

เห็นได้ว่าMSE เริ่มต้นจะเจอตัวที่ดีกว่าหรือเท่ากันใน generation ที่ 1 และเขียนแบบนี้ไปเรื่อย ๆ จนกว่าจะเจอ MSE ที่ดีกว่าเป็นต้น

errorMSE.txt

จะเป็นไฟล์ที่เก็บค่าขนาดของ reservoir หรือ unit ที่ทำให้โมดูล minimal_esn.py เกิดค่า MSE เป็น inf โดยใช้ดาต้า MackeyGlass_t17.txt จะมีทั้งหมด 22 ค่าด้วยกันดังนี้

```
errorMSE.txt
2
16
17
18
19
20
21
22
23
26
27
29
35
40
42
113
137
142
145
208
288
345
```

โดยค่าเหล่านี้เกิดจากการรันเช็คค่าขนาดของ reservoir ในโมดูล minimal_esn.py โดยทำการเขียนลูปเช็คขนาด reservoir ตั้งแต่ 1 ถึง 1024

```
for i in range(1, 1025):
    resSize = i
```

รูปผนวก ที่ ก 19 ลูปในการเช็คค่า reservoir

```
diff = sum(np.square(data[trainLen + 1:trainLen + errorLen + 1] - Y[0, 0:errorLen]))
if not np.isnan(diff).any() and not np.isinf(diff).any():
    mse = float(diff / errorLen)
else:
    with open('errorMSE.txt', 'a') as file:
        file.write(f"{resSize}\n")
```

รูปผนวก ที่ ก 20 เงื่อนไขในการเช็คหาค่า inf

โดยจะใช้เงื่อนไข if else ในการเช็คหาค่า inf โดยจะเช็คค่า nan และ inf ถ้าไม่เจอจะเก็บเข้าไปที่ตัวแปร mse ถ้ามีตัวใดตัวนึงตรงเงื่อนไงก็จะพิมพ์ค่าลงไปในไฟล์ errorMSE.txt

การเก็บค่าสถิติลงตาราง ในหัวข้อ 6.1.2

ในการเก็บค่าสถิตินั้นจะเก็บจากค่า MSE ที่ได้ทำการหาค่าที่ดีที่สุดมาจากในแต่ละการรันโดยมี 30 รัน ซึ่งจะมีค่า MSE 30 ตัวด้วยกันโดยจะมีกระบวนการดังนี้

- ในการเก็บผลของแต่รันนั้นเนื่องด้วยมีจำนวนรุ่น ในการเก็บอยู่ 101 รุ่นด้วยกันคือตัวเริ่มต้น 1 ตัว และรุ่นที่ได้ทำการส่งต่อจำนวน 100 รุ่น เราจะทำการหาค่า MSE ที่ดีที่สุดมาจำนวน 1 ค่าในการรันจาก 101 ค่า
- หลังทำการรันครบเราจะหาค่าที่ดีที่สุดของ MSE แต่ละรันมาได้ 30 ค่าด้วยกันโดยจะนำ 30 ค่านี้ไปหาค่าทางสถิติ

จะไปประยุกต์กับโค้ดดังนี้

โดยการเก็บของเรานั้นจะทำการรันให้เสร็จ 30 รันด้วยกันซึ่งได้รับค่า MSE มาทั้งหมด 3030 ค่า เก็บไว้ที่ไฟล์ bestMSE.txt รันไฟล์ findbestFOR30.py

```
import numpy as np
bestInGeneration = np.loadtxt('bestMSE.txt')
#bestInGenerationRE = np.loadtxt('bestReservoir.txt')

with open('bestGeneration30.txt', 'a') as file:

    smallest_Gen = float('inf')  # Initialize with infinity
    gen = 0  # Initialize generation index
    g=0
    c=0
    v=0
    for j in range(30):

        for i in range(g,g+101):
            if bestInGeneration[i] <= smallest_Gen:
                smallest_Gen = bestInGeneration[i]
                v=c
            c+=1
        file.write(F"{smallest_Gen}\n")
        gen = 0
        #file.write("\n")

        g+=101
        smallest_Gen = float('inf')
```

รูปผนวก ที่ ก 21 findbestFOR30.py

โดยไฟล์นี้จะเป็นการหาค่าที่ดีที่สุดของแต่ละการรันโดยทำการเช็คที่ละ 101 ค่า 30 ครั้งด้วยกัน จากนั้นก็จะเขียนค่า MSE ลงไปยังไฟล์ bestGeneration30.txt จำนวน 30 ค่าด้วยกัน

- หาค่าสถิติจากไฟล์ bestGeneration30.txt โดยหาจากฟังก์ชันของ numpy และนำค่าเหล่านี้ไปเขียนลงในตาราง

```
import numpy as np

best_mse = np.loadtxt('bestGeneration30.txt')

# Calculate statistics for bestMSE.txt
avg_mse = np.mean(best_mse)
stdev_mse = np.std(best_mse)
min_mse = np.min(best_mse)
max_mse = np.max(best_mse)
# Print the statistics
print("Statistics for bestMSE.txt:")
print(f"Average: {avg_mse}, Standard Deviation: {stdev_mse}, Minimum: {min_mse}, Maximum: {max_mse}")
```

รูปผนวก ที่ ก 22 หาค่าสถิติโดยใช้ numpy